

(12) **United States Patent**
Li et al.

(10) **Patent No.:** **US 12,418,651 B2**
(45) **Date of Patent:** **Sep. 16, 2025**

(54) **CABAC INHERITED CONTEXT INITIALIZATION**

(71) Applicant: **Tencent America LLC**, Palo Alto, CA (US)

(72) Inventors: **Xiang Li**, Saratoga, CA (US); **Guichun Li**, San Jose, CA (US); **Shan Liu**, San Jose, CA (US)

(73) Assignee: **Tencent America LLC**, Palo Alto, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 188 days.

(21) Appl. No.: **17/940,981**

(22) Filed: **Sep. 8, 2022**

(65) **Prior Publication Data**
US 2023/0336728 A1 Oct. 19, 2023

Related U.S. Application Data

(60) Provisional application No. 63/331,940, filed on Apr. 18, 2022.

(51) **Int. Cl.**
H04N 19/00 (2014.01)
H04N 19/105 (2014.01)
H04N 19/13 (2014.01)
H04N 19/172 (2014.01)
H04N 19/174 (2014.01)
(Continued)

(52) **U.S. Cl.**
CPC **H04N 19/13** (2014.11); **H04N 19/105** (2014.11); **H04N 19/172** (2014.11); **H04N 19/174** (2014.11); **H04N 19/177** (2014.11); **H04N 19/70** (2014.11)

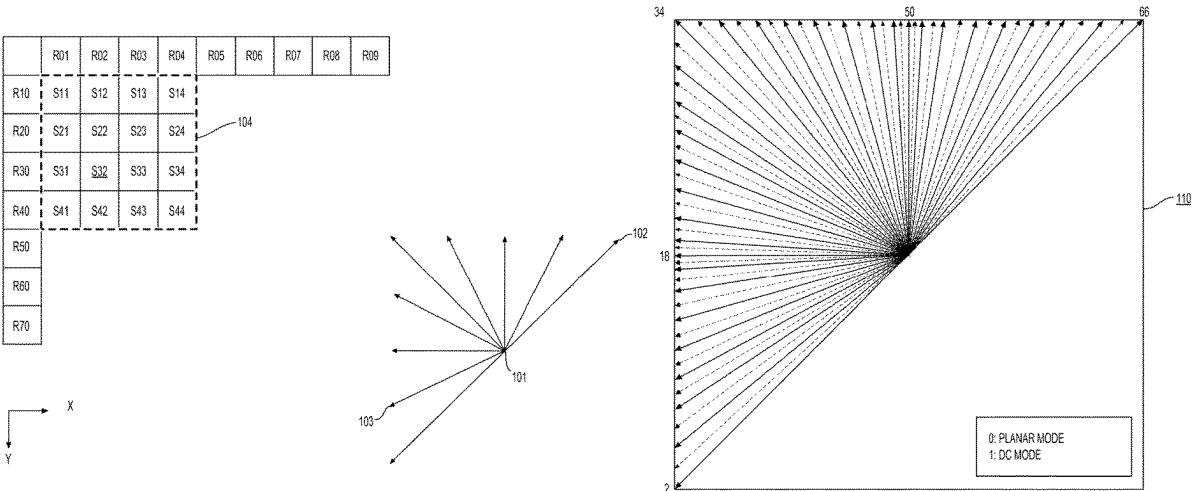
(58) **Field of Classification Search**
CPC H04N 19/13; H04N 19/105; H04N 19/172; H04N 19/174; H04N 19/177; H04N 19/70
See application file for complete search history.

(56) **References Cited**
U.S. PATENT DOCUMENTS
2015/0334425 A1* 11/2015 He H04N 19/174 375/240.03
2016/0353110 A1 12/2016 Zhang et al.
(Continued)

OTHER PUBLICATIONS
International Search Report and Written Opinion issued Dec. 14, 2022 in Application No. PCT/US2022/76274, 11 pages.
(Continued)
Primary Examiner — Zhihan Zhou
(74) *Attorney, Agent, or Firm* — ArentFox Schiff LLP

(57) **ABSTRACT**
Aspects of the disclosure provide methods and apparatuses for video encoding/decoding. In some examples, an apparatus for video decoding includes receiving circuitry and processing circuitry. The processing circuitry determines, a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for decoding a current portion of a video sequence carried by a bitstream. The processing circuitry decodes, from the bitstream, a syntax value that indicates a decoded portion in the video sequence for a context inheriting, and initializes context models for the CABAC based on stored context states associated with the decoded portion. Further, the processing circuitry decodes, the current portion from the bitstream according to the context models that are initialized based on the stored context states associated with the decoded portion.

16 Claims, 14 Drawing Sheets



(51) **Int. Cl.**

H04N 19/177 (2014.01)

H04N 19/70 (2014.01)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2021/0014480 A1 * 1/2021 George H04N 19/105
2025/0008107 A1 * 1/2025 Xiu H04N 19/119

OTHER PUBLICATIONS

High Efficiency Video Coding, Rec. ITU-T H.265 v4 Dec. 2016, pp. 1-664.

B. Bross, S. Liu, et al., "Versatile Video Coding Editorial Refinements on Draft 10", JVET-T2001, ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Oct. 2020, pp. 1-511.

J. Chen, Et. Al., "Algorithm description for Versatile Video Coding and Test Model 10 (VTM 10)", JVET-S2002, ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Jul. 2020, pp. 1-97.

X. Xiu, W. Chen, C.-W. Kuo, H.-J. Jhu, N. Yan, X. Wang, "EE2-Test4.1: Improved probability estimation for CABAC", IVET-Z0134, ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Apr. 2022, pp. 1-5.

* cited by examiner

	R01	R02	R03	R04	R05	R06	R07	R08	R09
R10	S11	S12	S13	S14	104				
R20	S21	S22	S23	S24					
R30	S31	S32	S33	S34					
R40	S41	S42	S43	S44					
R50									
R60									
R70									

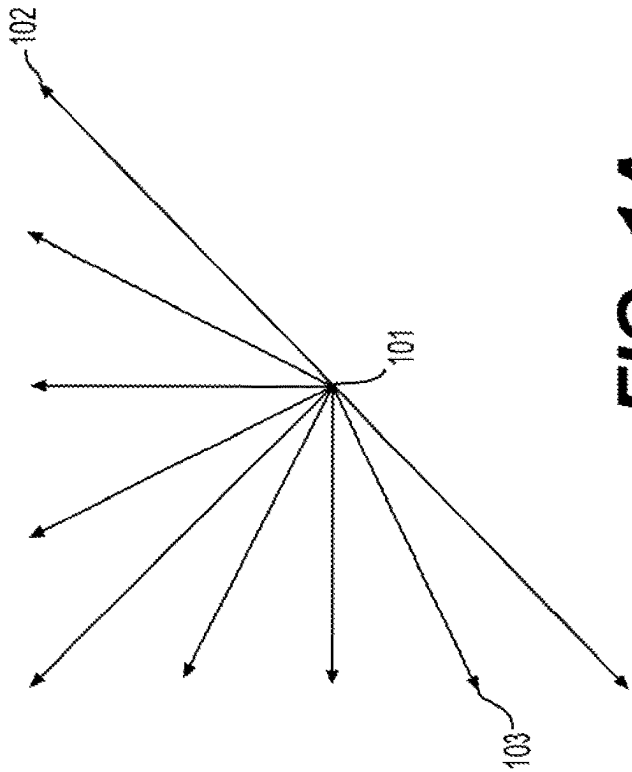
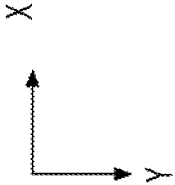
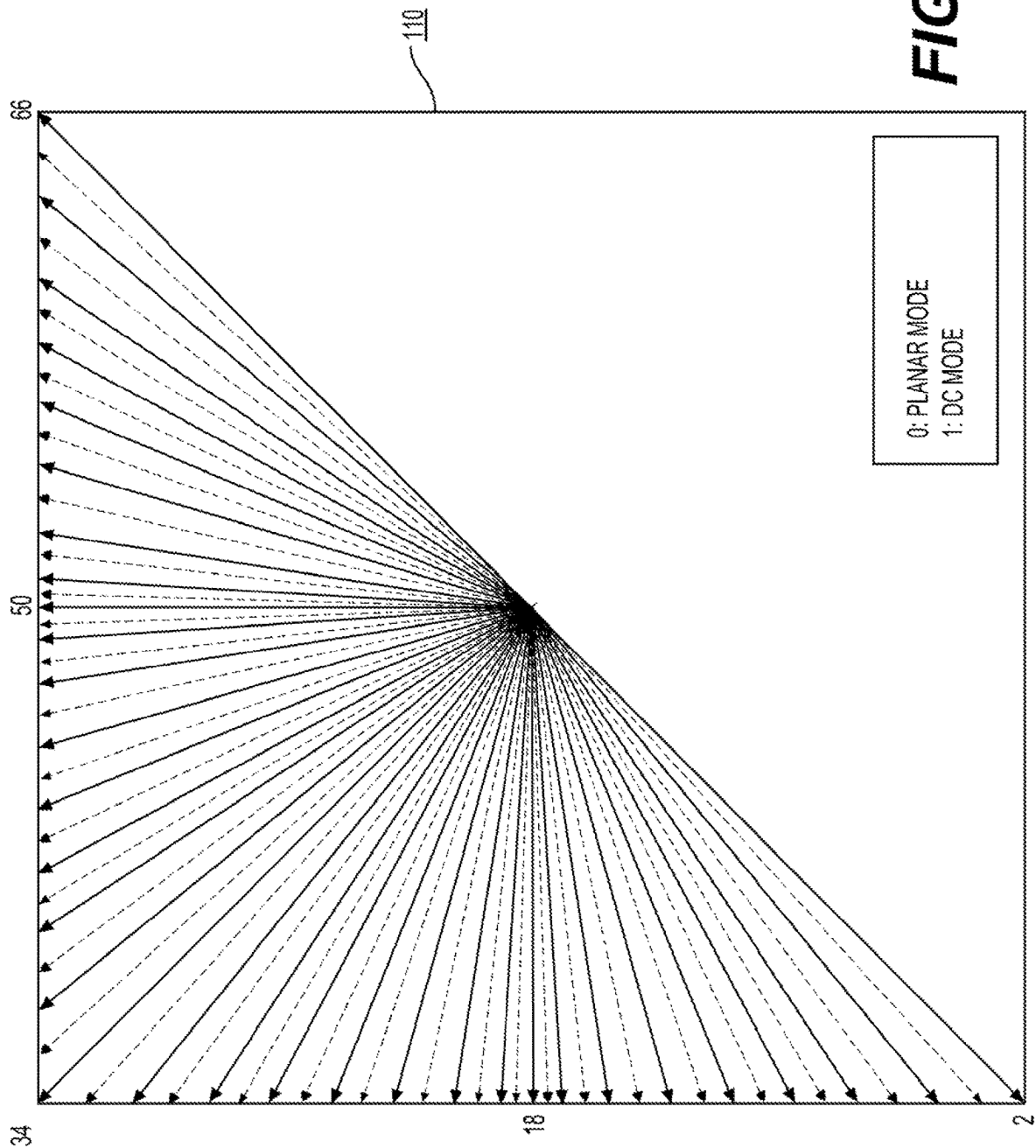


FIG. 1A





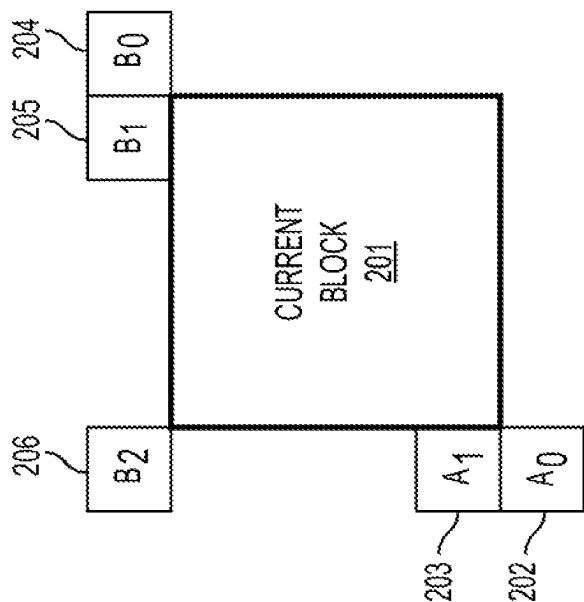
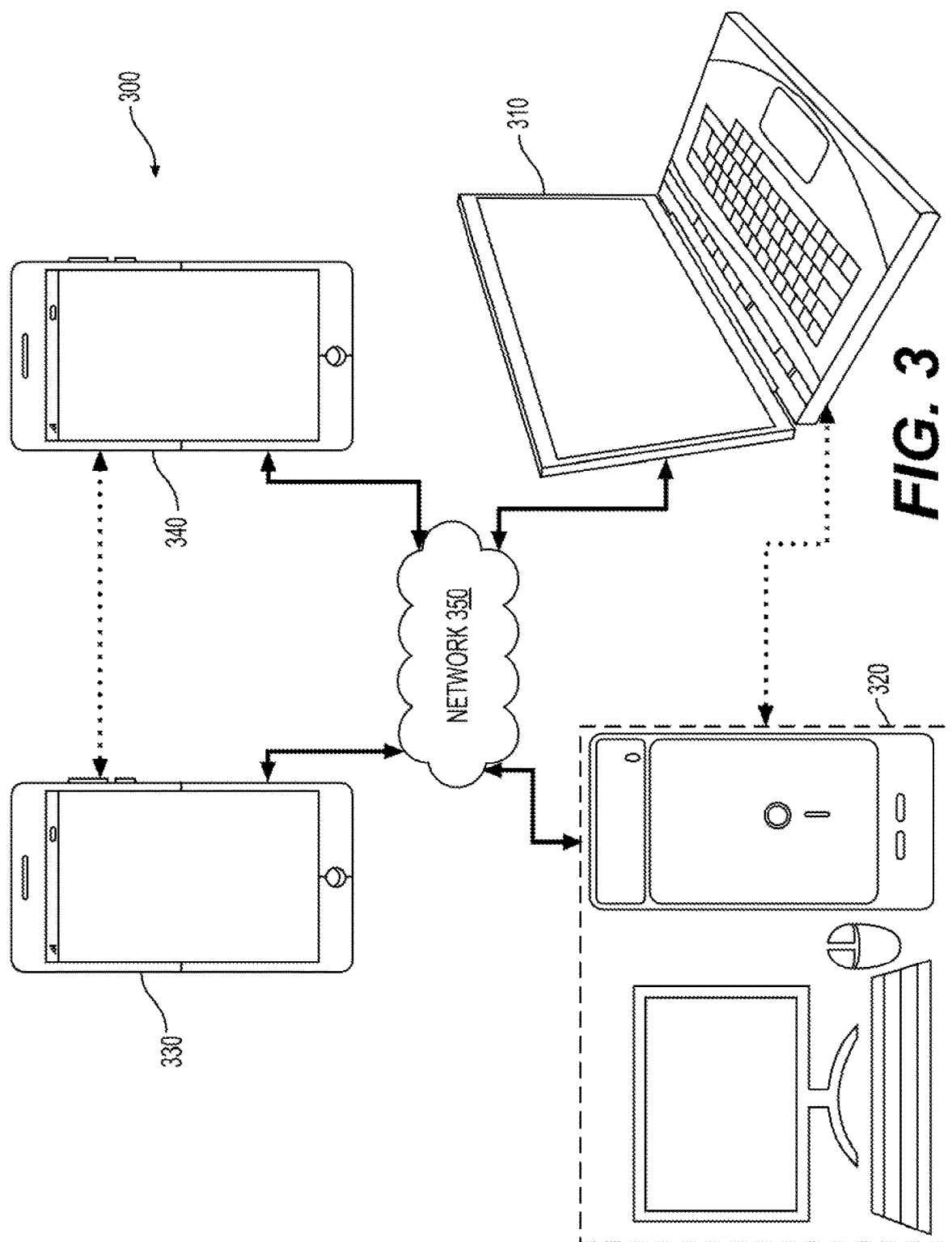


FIG. 2
(Related Art)



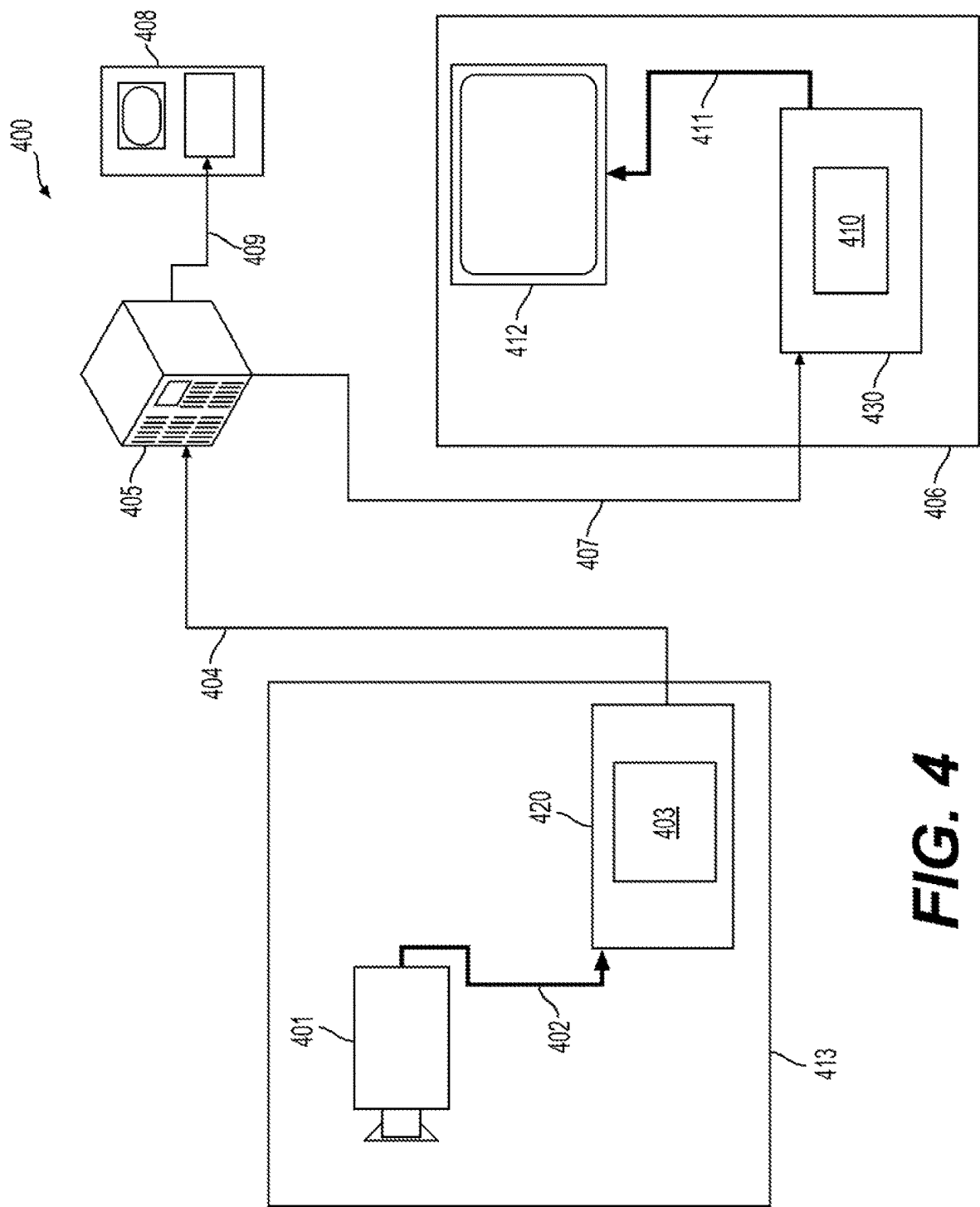


FIG. 4

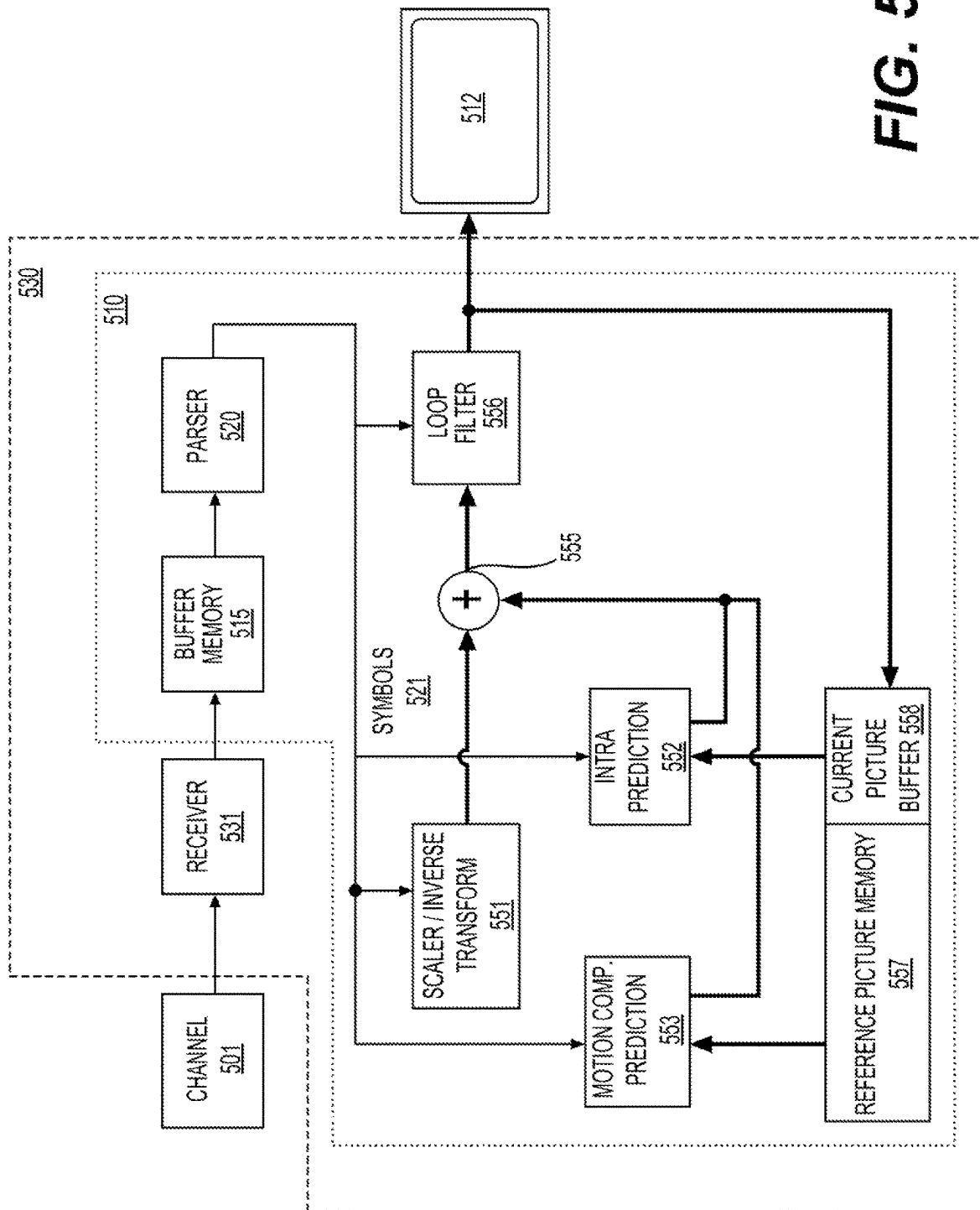
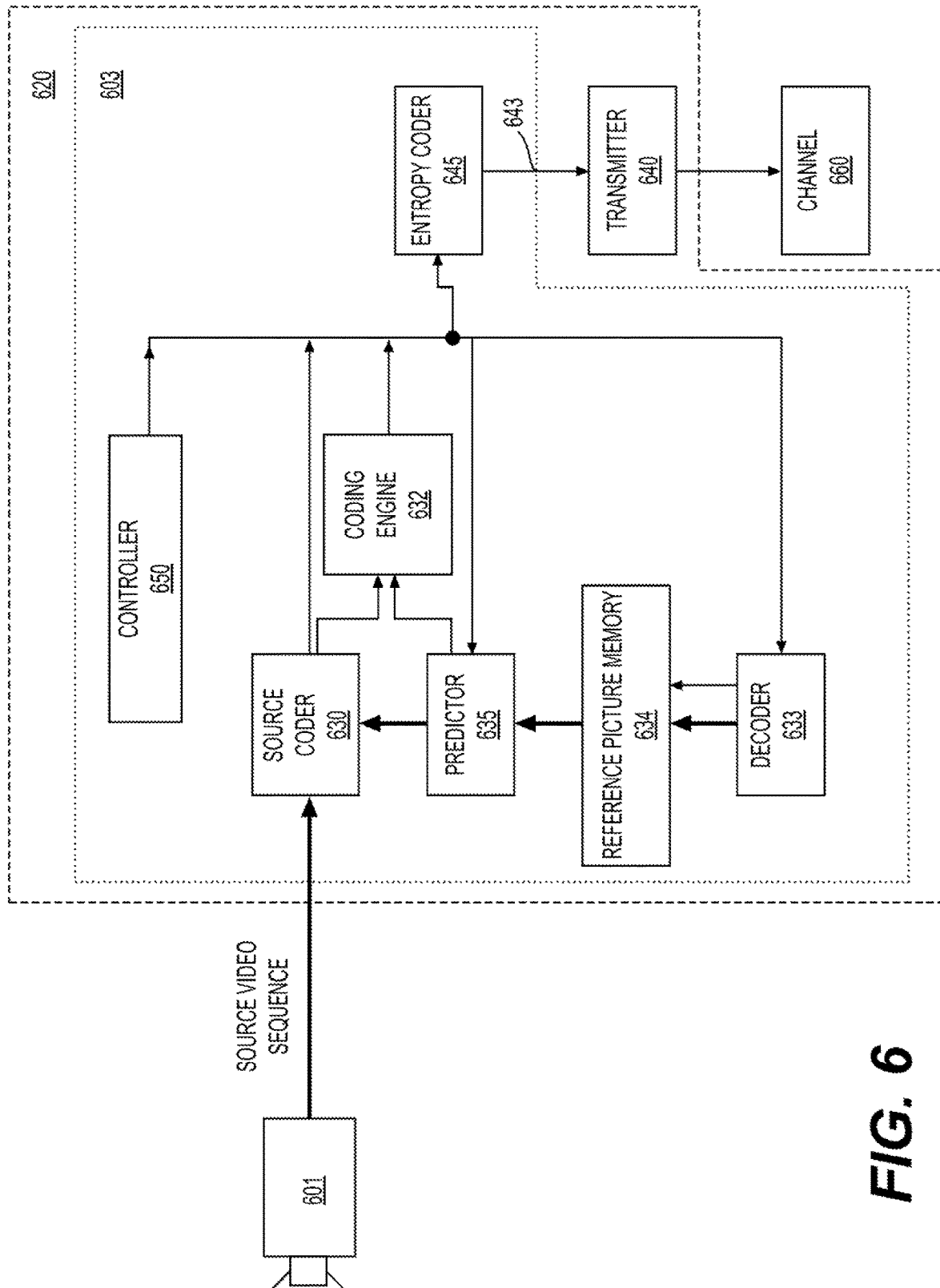


FIG. 5

**FIG. 6**

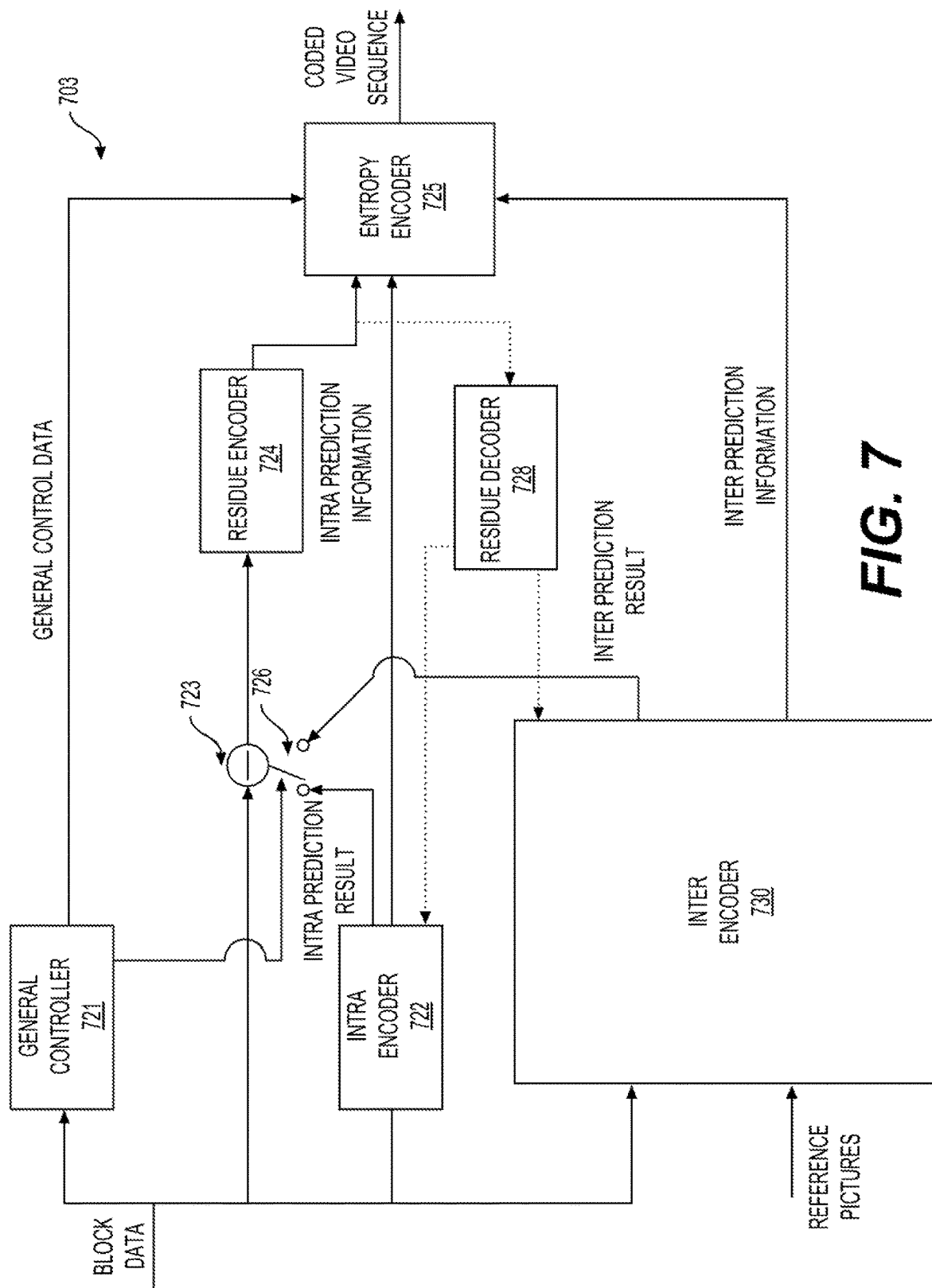


FIG. 7

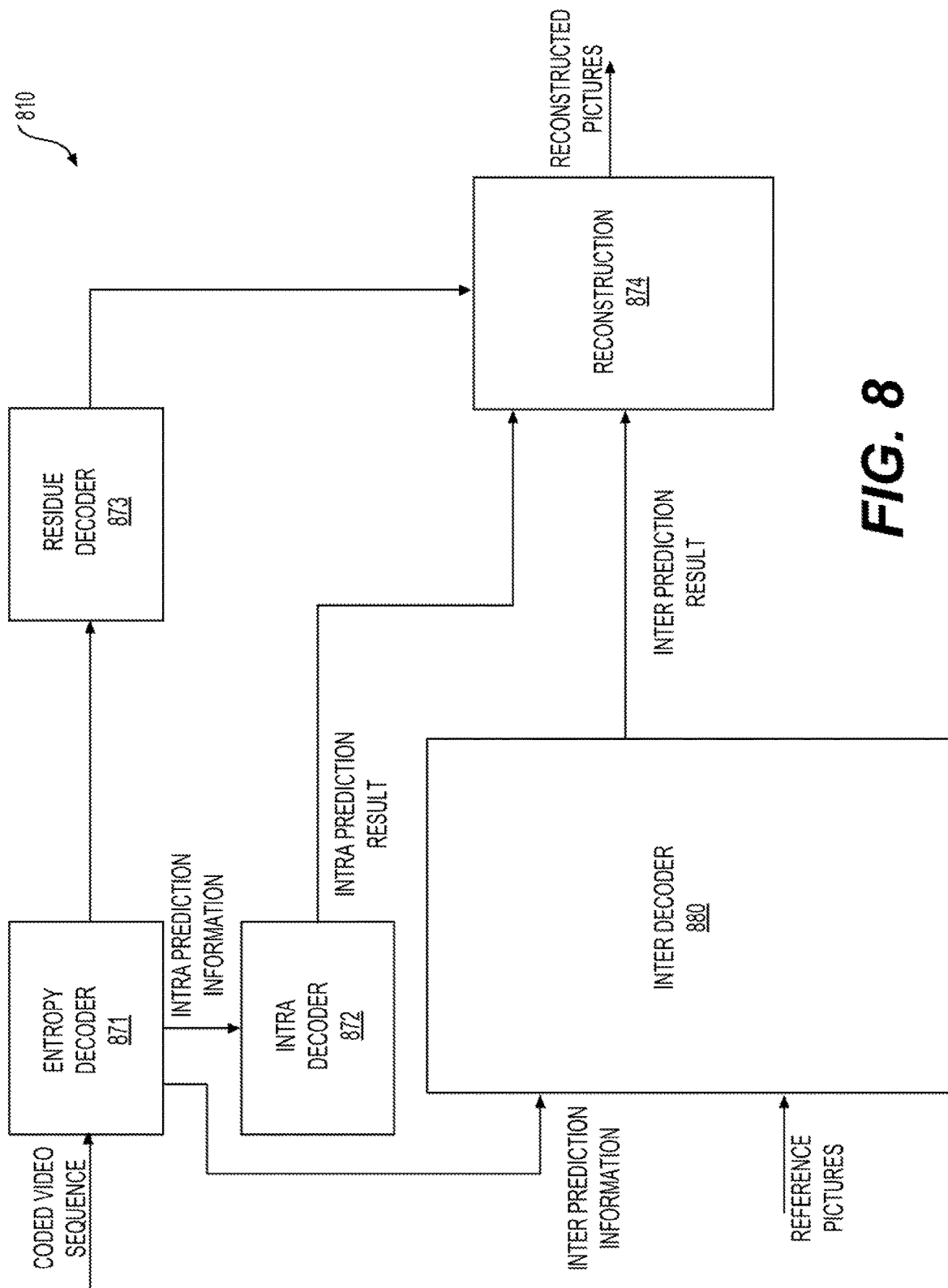
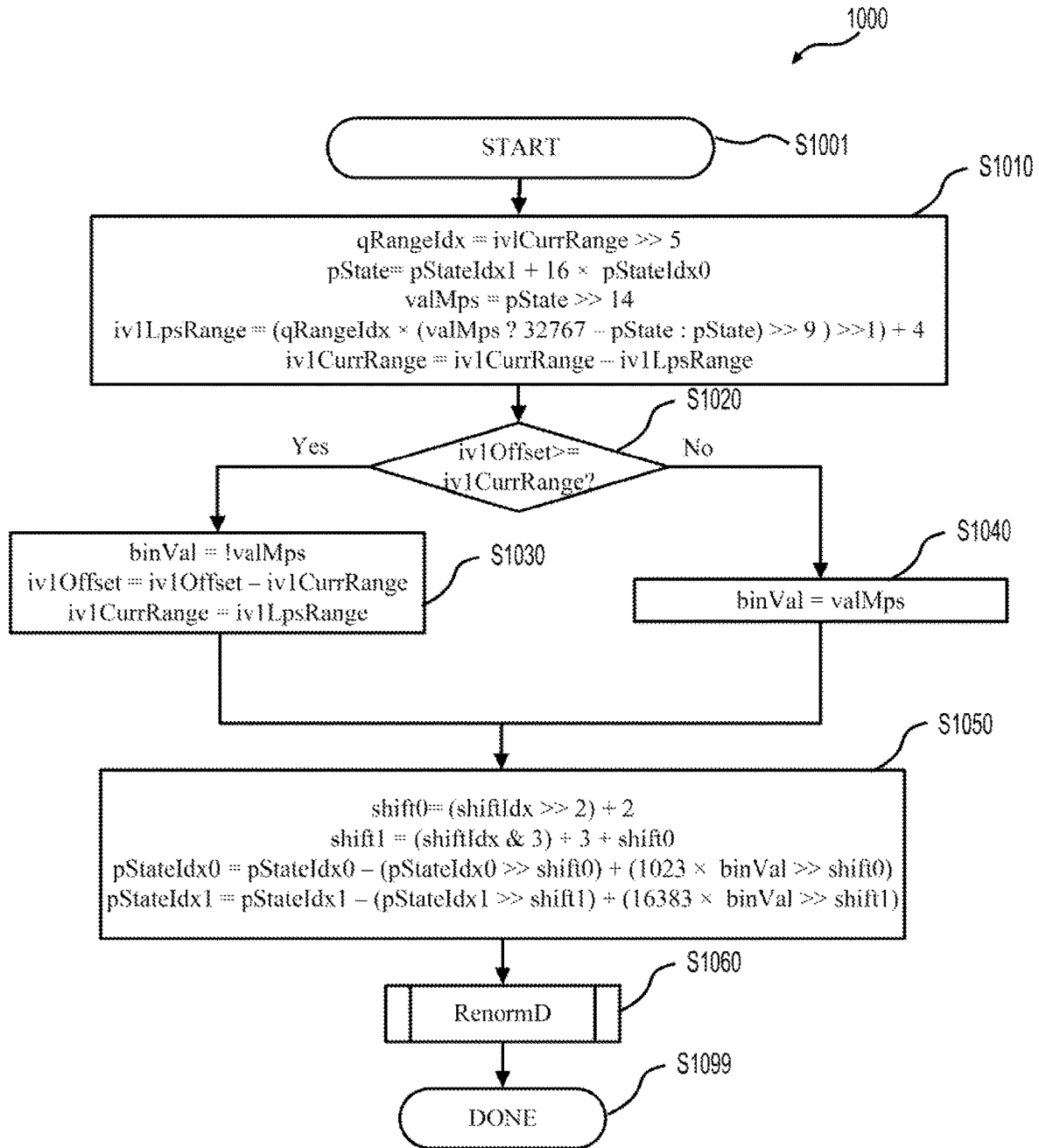


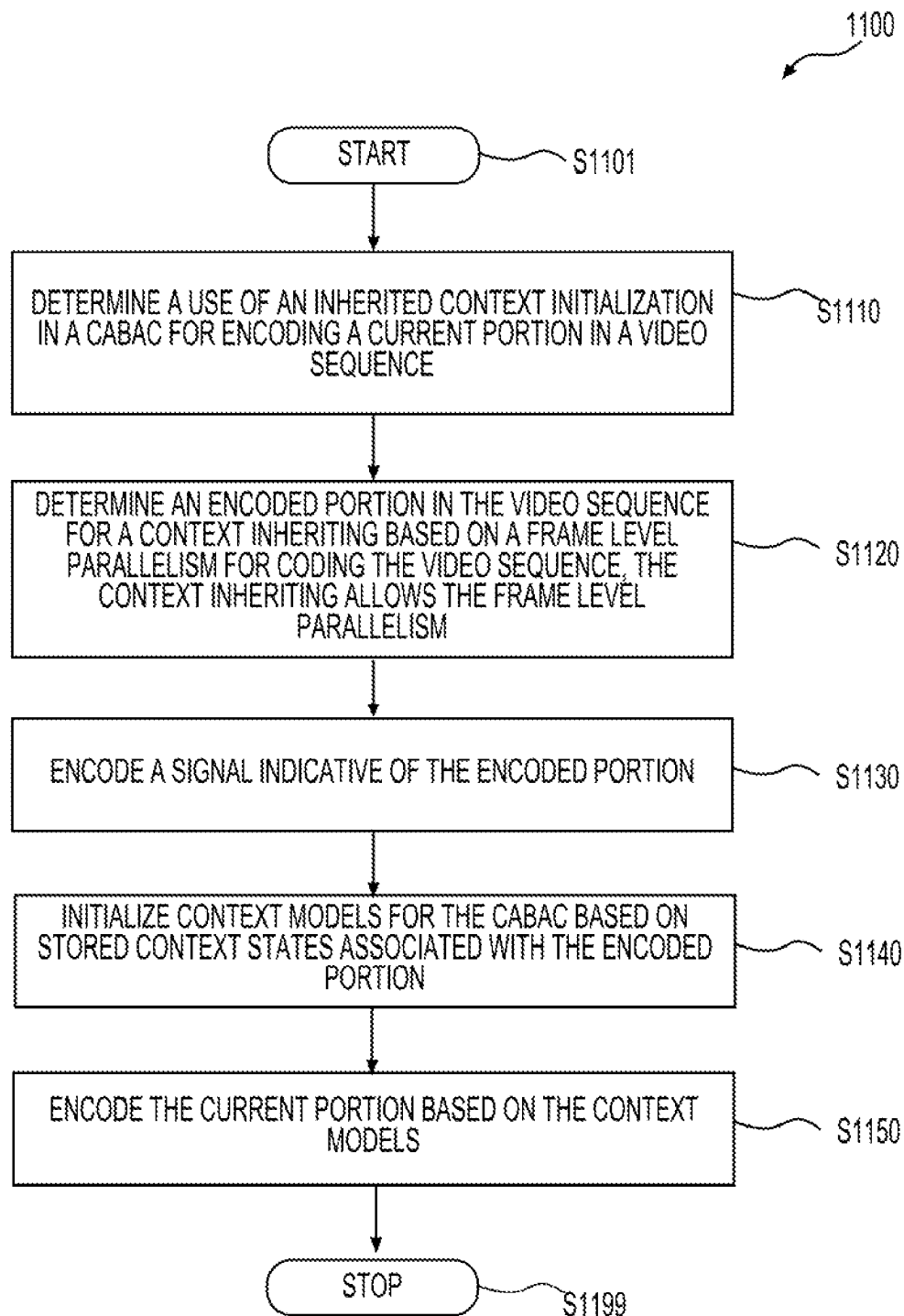
FIG. 8

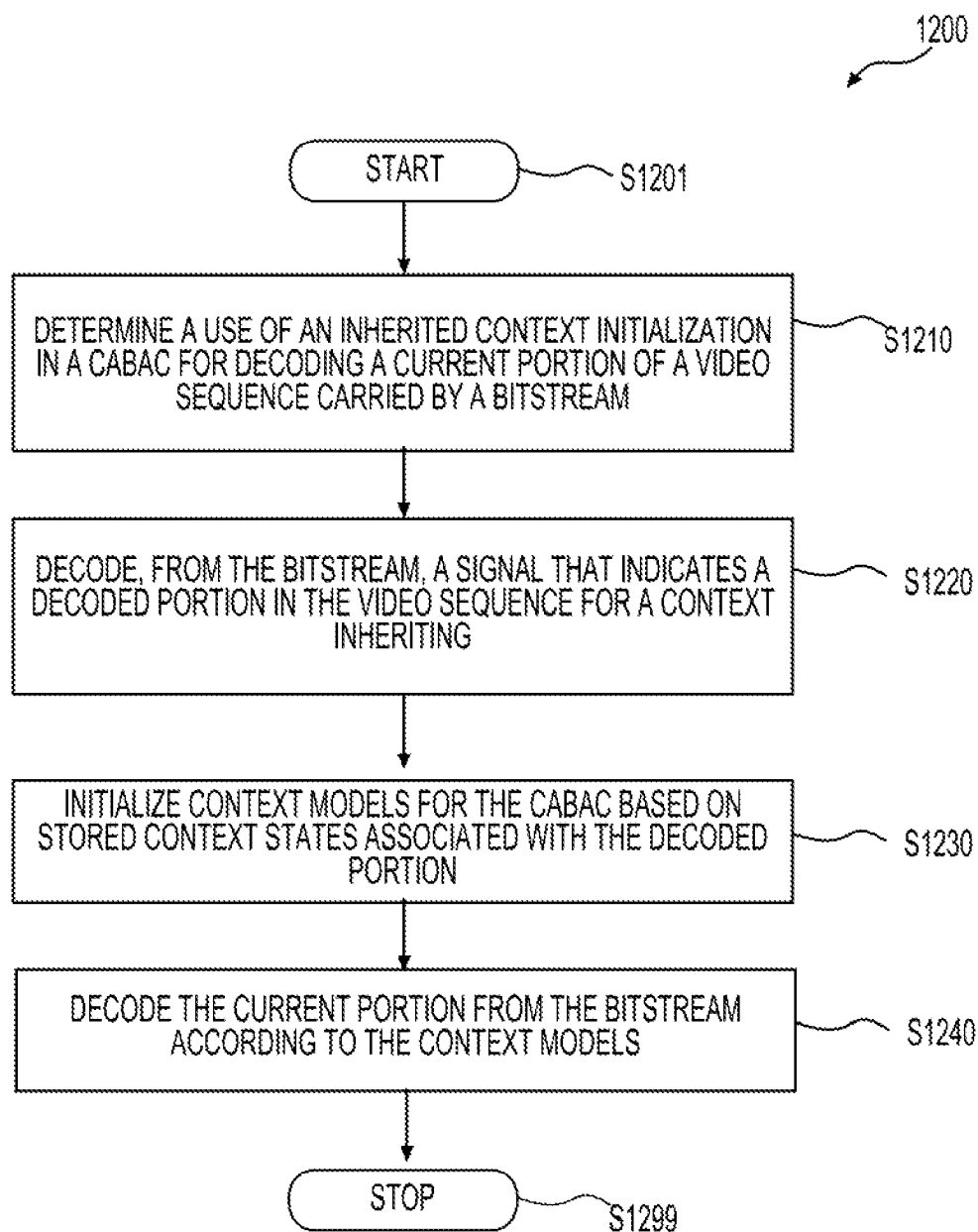
910

Probability State	LPS Range			
	Segment 0	Segment 1	Segment 2	Segment 3
...				
9	90	110	130	150
10	85	104	123	142
11	81	99	117	135
12	77	94	111	128
...				

FIG. 9

**FIG. 10**

**FIG. 11**

**FIG. 12**

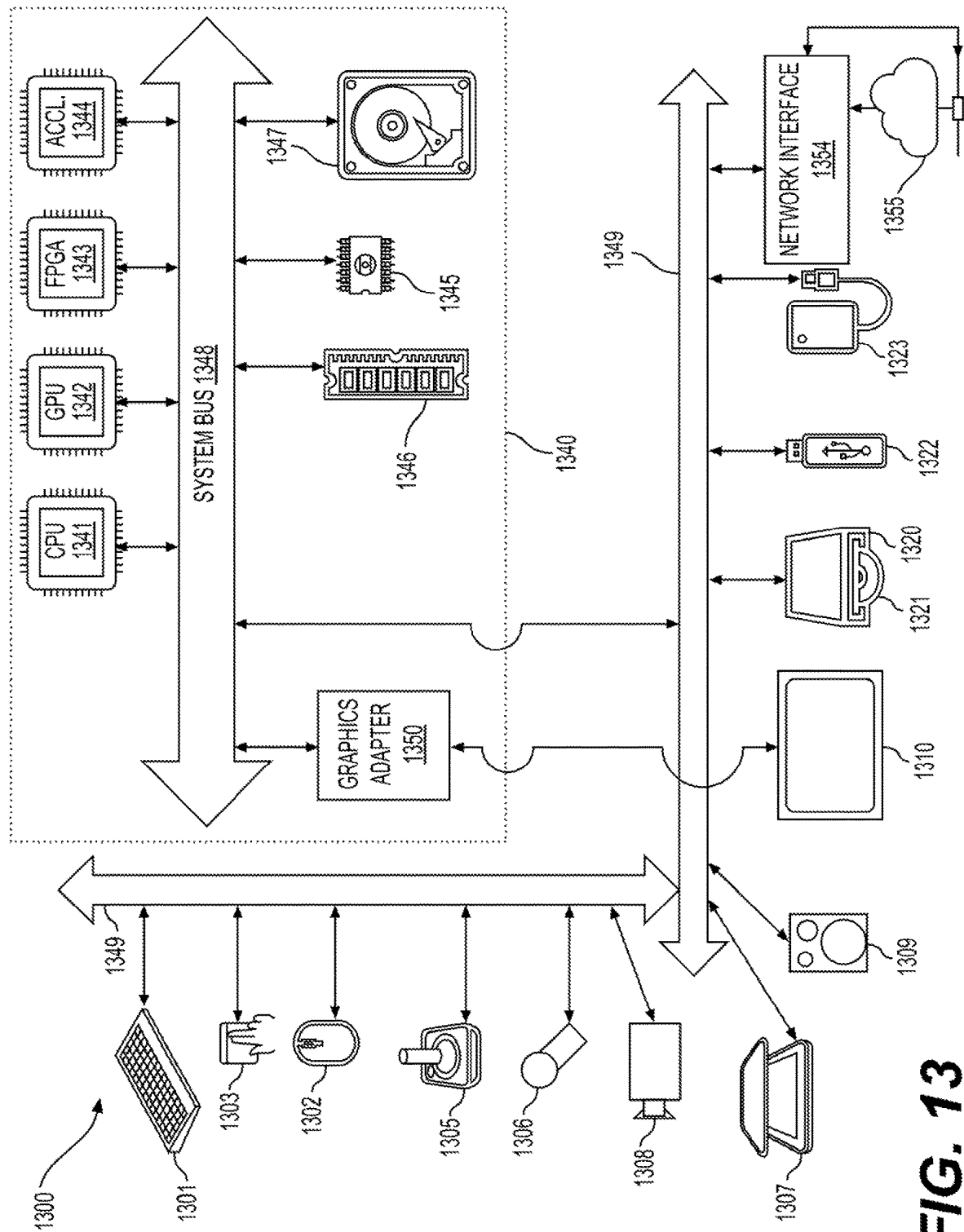


FIG. 13

CABAC INHERITED CONTEXT INITIALIZATION

INCORPORATION BY REFERENCE

This present disclosure claims the benefit of priority to U.S. Provisional Application No. 63/331,940, "Method and Apparatus for Context-Based Adaptive Binary Arithmetic Coding Inherited Context Initialization" filed on Apr. 18, 2022, which is incorporated by reference herein in its entirety.

TECHNICAL FIELD

The present disclosure describes embodiments generally related to video coding.

BACKGROUND

The background description provided herein is for the purpose of generally presenting the context of the disclosure. Work of the presently named inventors, to the extent the work is described in this background section, as well as aspects of the description that may not otherwise qualify as prior art at the time of filing, are neither expressly nor impliedly admitted as prior art against the present disclosure.

Uncompressed digital video can include a series of pictures, each picture having a spatial dimension of, for example, 1920×1080 luminance samples and associated chrominance samples. The series of pictures can have a fixed or variable picture rate (informally also known as frame rate), of, for example 60 pictures per second or 60 Hz. Uncompressed video has specific bitrate requirements. For example, 1080p60 4:2:0 video at 8 bit per sample (1920×1080 luminance sample resolution at 60 Hz frame rate) requires close to 1.5 Gbit/s bandwidth. An hour of such video requires more than 600 GBytes of storage space.

One purpose of video coding and decoding can be the reduction of redundancy in the input video signal, through compression. Compression can help reduce the aforementioned bandwidth and/or storage space requirements, in some cases by two orders of magnitude or more. Both lossless compression and lossy compression, as well as a combination thereof can be employed. Lossless compression refers to techniques where an exact copy of the original signal can be reconstructed from the compressed original signal. When using lossy compression, the reconstructed signal may not be identical to the original signal, but the distortion between original and reconstructed signals is small enough to make the reconstructed signal useful for the intended application. In the case of video, lossy compression is widely employed. The amount of distortion tolerated depends on the application; for example, users of certain consumer streaming applications may tolerate higher distortion than users of television distribution applications. The compression ratio achievable can reflect that: higher allowable/tolerable distortion can yield higher compression ratios.

A video encoder and decoder can utilize techniques from several broad categories, including, for example, motion compensation, transform, quantization, and entropy coding.

Video codec technologies can include techniques known as intra coding. In intra coding, sample values are represented without reference to samples or other data from previously reconstructed reference pictures. In some video codecs, the picture is spatially subdivided into blocks of samples. When all blocks of samples are coded in intra

mode, that picture can be an intra picture. Intra pictures and their derivations such as independent decoder refresh pictures, can be used to reset the decoder state and can, therefore, be used as the first picture in a coded video bitstream and a video session, or as a still image. The samples of an intra block can be exposed to a transform, and the transform coefficients can be quantized before entropy coding. Intra prediction can be a technique that minimizes sample values in the pre-transform domain. In some cases, the smaller the DC value after a transform is, and the smaller the AC coefficients are, the fewer the bits that are required at a given quantization step size to represent the block after entropy coding.

Traditional intra coding such as known from, for example MPEG-2 generation coding technologies, does not use intra prediction. However, some newer video compression technologies include techniques that attempt, from, for example, surrounding sample data and/or metadata obtained during the encoding and/or decoding of spatially neighboring, and preceding in decoding order, blocks of data. Such techniques are henceforth called "intra prediction" techniques. Note that in at least some cases, intra prediction is using reference data only from the current picture under reconstruction and not from reference pictures.

There can be many different forms of intra prediction. When more than one of such techniques can be used in a given video coding technology, the technique in use can be coded in an intra prediction mode. In certain cases, modes can have submodes and/or parameters, and those can be coded individually or included in the mode codeword. Which codeword to use for a given mode, submode, and/or parameter combination can have an impact in the coding efficiency gain through intra prediction, and so can the entropy coding technology used to translate the codewords into a bitstream.

A certain mode of intra prediction was introduced with H.264, refined in H.265, and further refined in newer coding technologies such as joint exploration model (JEM), versatile video coding (VVC), and benchmark set (BMS). A predictor block can be formed using neighboring sample values belonging to already available samples. Sample values of neighboring samples are copied into the predictor block according to a direction. A reference to the direction in use can be coded in the bitstream or may itself be predicted.

Referring to FIG. 1A, depicted in the lower right is a subset of nine predictor directions known from H.265's 33 possible predictor directions (corresponding to the 33 angular modes of the 35 intra modes). The point where the arrows converge (**101**) represents the sample being predicted. The arrows represent the direction from which the sample is being predicted. For example, arrow (**102**) indicates that sample (**101**) is predicted from a sample or samples to the upper right, at a 45 degree angle from the horizontal. Similarly, arrow (**103**) indicates that sample (**101**) is predicted from a sample or samples to the lower left of sample (**101**), in a 22.5 degree angle from the horizontal.

Still referring to FIG. 1A, on the top left there is depicted a square block (**104**) of 4×4 samples (indicated by a dashed, boldface line). The square block (**104**) includes 16 samples, each labelled with an "S", its position in the Y dimension (e.g., row index) and its position in the X dimension (e.g., column index). For example, sample **S21** is the second sample in the Y dimension (from the top) and the first (from the left) sample in the X dimension. Similarly, sample **S44** is the fourth sample in block (**104**) in both the Y and X dimensions. As the block is 4×4 samples in size, **S44** is at the

bottom right. Further shown are reference samples that follow a similar numbering scheme. A reference sample is labelled with an R, its Y position (e.g., row index) and X position (column index) relative to block (104). In both H.264 and H.265, prediction samples neighbor the block under reconstruction; therefore no negative values need to be used.

Intra picture prediction can work by copying reference sample values from the neighboring samples as appropriated by the signaled prediction direction. For example, assume the coded video bitstream includes signaling that, for this block, indicates a prediction direction consistent with arrow (102)—that is, samples are predicted from a prediction sample or samples to the upper right, at a 45 degree angle from the horizontal. In that case, samples S41, S32, S23, and S14 are predicted from the same reference sample R05. Sample S44 is then predicted from reference sample R08.

In certain cases, the values of multiple reference samples may be combined, for example through interpolation, in order to calculate a reference sample; especially when the directions are not evenly divisible by 45 degrees.

The number of possible directions has increased as video coding technology has developed. In H.264 (year 2003), nine different direction could be represented. That increased to 33 in H.265 (year 2013), and JEM/VVC/BMS, at the time of disclosure, can support up to 65 directions. Experiments have been conducted to identify the most likely directions, and certain techniques in the entropy coding are used to represent those likely directions in a small number of bits, accepting a certain penalty for less likely directions. Further, the directions themselves can sometimes be predicted from neighboring directions used in neighboring, already decoded, blocks.

FIG. 1B shows a schematic (110) that depicts 65 intra prediction directions according to JEM to illustrate the increasing number of prediction directions over time.

The mapping of intra prediction directions bits in the coded video bitstream that represent the direction can be different from video coding technology to video coding technology; and can range, for example, from simple direct mappings of prediction direction to intra prediction mode, to codewords, to complex adaptive schemes involving most probable modes, and similar techniques. In all cases, however, there can be certain directions that are statistically less likely to occur in video content than certain other directions. As the goal of video compression is the reduction of redundancy, those less likely directions will, in a well working video coding technology, be represented by a larger number of bits than more likely directions.

Video coding and decoding can be performed using inter-picture prediction with motion compensation. Motion compensation can be a lossy compression technique and can relate to techniques where a block of sample data from a previously reconstructed picture or part thereof (reference picture), after being spatially shifted in a direction indicated by a motion vector (MV henceforth), is used for the prediction of a newly reconstructed picture or picture part. In some cases, the reference picture can be the same as the picture currently under reconstruction. MVs can have two dimensions X and Y, or three dimensions, the third being an indication of the reference picture in use (the latter, indirectly, can be a time dimension).

In some video compression techniques, an MV applicable to a certain area of sample data can be predicted from other MVs, for example from those related to another area of sample data spatially adjacent to the area under reconstruction, and preceding that MV in decoding order. Doing so can

substantially reduce the amount of data required for coding the MV, thereby removing redundancy and increasing compression. MV prediction can work effectively, for example, because when coding an input video signal derived from a camera (known as natural video) there is a statistical likelihood that areas larger than the area to which a single MV is applicable move in a similar direction and, therefore, can in some cases be predicted using a similar motion vector derived from MVs of neighboring area. That results in the MV found for a given area to be similar or the same as the MV predicted from the surrounding MVs, and that in turn can be represented, after entropy coding, in a smaller number of bits than what would be used if coding the MV directly. In some cases, MV prediction can be an example of lossless compression of a signal (namely: the MVs) derived from the original signal (namely: the sample stream). In other cases, MV prediction itself can be lossy, for example because of rounding errors when calculating a predictor from several surrounding MVs.

Various MV prediction mechanisms are described in H.265/HEVC (ITU-T Rec. H.265, “High Efficiency Video Coding”, December 2016). Out of the many MV prediction mechanisms that H.265 offers, described here is a technique henceforth referred to as “spatial merge”.

Referring to FIG. 2, a current block (201) comprises samples that have been found by the encoder during the motion search process to be predictable from a previous block of the same size that has been spatially shifted. Instead of coding that MV directly, the MV can be derived from metadata associated with one or more reference pictures, for example from the most recent (in decoding order) reference picture, using the MV associated with either one of five surrounding samples, denoted A0, A1, and B0, B1, B2 (202 through 206, respectively). In H.265, the MV prediction can use predictors from the same reference picture that the neighboring block is using.

SUMMARY

Aspects of the disclosure provide methods and apparatuses for video encoding/decoding. In some examples, an apparatus for video decoding includes receiving circuitry and processing circuitry. The processing circuitry determines, a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for decoding a current portion of a video sequence carried by a bitstream. The processing circuitry decodes, from the bitstream, a syntax value that indicates a decoded portion in the video sequence for a context inheriting, and initializes context models for the CABAC based on stored context states associated with the decoded portion. Further, the processing circuitry decodes, the current portion from the bitstream according to the context models that are initialized based on the stored context states associated with the decoded portion.

In an example, the syntax value indicates a decoded picture for the context inheriting. In another example, the syntax value indicates a decoded slice for the context inheriting. In some examples, the syntax value is decoded from at least one of a picture parameter set (PPS), a picture header, and a slice header.

In some examples, the decoded portion is a reference that is used for the decoding of the current portion.

In some examples, the processing circuitry decodes the syntax value that indicates at least a first decoded portion and a second decoded portion and initializes the context models for the CABAC according to a weighted average of

5

probability values indicated by at least first context states associated with the first decoded portion and second context states associated with the second decoded portion.

In some examples, the processing circuitry can decode a first weighting factor associated with the first decoded portion and a second weighting factor associated with the second decoded portion. The processing circuitry calculates the weighted average by applying the first weighting factor on first probability values indicated by the first context states associated with the first decoded portion, and applying the second weighting factor on second probability values indicated by the second context states associated with the second decoded portion.

In some examples, in response to the bitstream carrying no weighting factor associated with the first decoded portion, the processing circuitry determines to use zero weighting factor or a default weighting factor associated with the first decoded portion.

In some examples, the processing circuitry decodes, from the bitstream, an indicator at a group of pictures (GOP) level. The indicator indicates one or more pictures that are allowed for the context inheriting. The processing circuitry prioritizes a decoding of a first picture in the one or more pictures before a decoding of a second picture not in the one or more pictures. In an example, the processing circuitry decodes the indicator from a reference picture list structure syntax.

Aspects of the disclosure also provide a non-transitory computer-readable medium storing instructions which when executed by a computer for video decoding cause the computer to perform the method for video decoding.

BRIEF DESCRIPTION OF THE DRAWINGS

Further features, the nature, and various advantages of the disclosed subject matter will be more apparent from the following detailed description and the accompanying drawings in which:

FIG. 1A is a schematic illustration of an exemplary subset of intra prediction modes.

FIG. 1B is an illustration of exemplary intra prediction directions.

FIG. 2 is a schematic illustration of a current block and its surrounding spatial merge candidates in one example.

FIG. 3 is a schematic illustration of a simplified block diagram of a communication system (300) in accordance with an embodiment.

FIG. 4 is a schematic illustration of a simplified block diagram of a communication system (400) in accordance with an embodiment.

FIG. 5 is a schematic illustration of a simplified block diagram of a decoder in accordance with an embodiment.

FIG. 6 is a schematic illustration of a simplified block diagram of an encoder in accordance with an embodiment.

FIG. 7 shows a block diagram of an encoder in accordance with another embodiment.

FIG. 8 shows a block diagram of a decoder in accordance with another embodiment.

FIG. 9 shows a table of quantized values of least probable symbol (LPS) ranges in some examples.

FIG. 10 shows a flowchart outlining a process for decoding a single binary decision in some examples.

FIG. 11 shows a flow chart outlining a process according to some embodiment of the disclosure.

FIG. 12 shows a flow chart outlining another process according to some embodiment of the disclosure.

6

FIG. 13 is a schematic illustration of a computer system in accordance with an embodiment.

DETAILED DESCRIPTION OF EMBODIMENTS

FIG. 3 illustrates a simplified block diagram of a communication system (300) according to an embodiment of the present disclosure. The communication system (300) includes a plurality of terminal devices that can communicate with each other, via, for example, a network (350). For example, the communication system (300) includes a first pair of terminal devices (310) and (320) interconnected via the network (350). In the FIG. 3 example, the first pair of terminal devices (310) and (320) performs unidirectional transmission of data. For example, the terminal device (310) may code video data (e.g., a stream of video pictures that are captured by the terminal device (310)) for transmission to the other terminal device (320) via the network (350). The encoded video data can be transmitted in the form of one or more coded video bitstreams. The terminal device (320) may receive the coded video data from the network (350), decode the coded video data to recover the video pictures and display video pictures according to the recovered video data. Unidirectional data transmission may be common in media serving applications and the like.

In another example, the communication system (300) includes a second pair of terminal devices (330) and (340) that performs bidirectional transmission of coded video data that may occur, for example, during videoconferencing. For bidirectional transmission of data, in an example, each terminal device of the terminal devices (330) and (340) may code video data (e.g., a stream of video pictures that are captured by the terminal device) for transmission to the other terminal device of the terminal devices (330) and (340) via the network (350). Each terminal device of the terminal devices (330) and (340) also may receive the coded video data transmitted by the other terminal device of the terminal devices (330) and (340), and may decode the coded video data to recover the video pictures and may display video pictures at an accessible display device according to the recovered video data.

In the FIG. 3 example, the terminal devices (310), (320), (330) and (340) may be illustrated as servers, personal computers and smart phones but the principles of the present disclosure may be not so limited. Embodiments of the present disclosure find application with laptop computers, tablet computers, media players and/or dedicated video conferencing equipment. The network (350) represents any number of networks that convey coded video data among the terminal devices (310), (320), (330) and (340), including for example wireline (wired) and/or wireless communication networks. The communication network (350) may exchange data in circuit-switched and/or packet-switched channels. Representative networks include telecommunications networks, local area networks, wide area networks and/or the Internet. For the purposes of the present discussion, the architecture and topology of the network (350) may be immaterial to the operation of the present disclosure unless explained herein below.

FIG. 4 illustrates, as an example for an application for the disclosed subject matter, the placement of a video encoder and a video decoder in a streaming environment. The disclosed subject matter can be equally applicable to other video enabled applications, including, for example, video conferencing, digital TV, storing of compressed video on digital media including CD, DVD, memory stick and the like, and so on.

A streaming system may include a capture subsystem (413), that can include a video source (401), for example a digital camera, creating for example a stream of video pictures (402) that are uncompressed. In an example, the stream of video pictures (402) includes samples that are taken by the digital camera. The stream of video pictures (402), depicted as a bold line to emphasize a high data volume when compared to encoded video data (404) (or coded video bitstreams), can be processed by an electronic device (420) that includes a video encoder (403) coupled to the video source (401). The video encoder (403) can include hardware, software, or a combination thereof to enable or implement aspects of the disclosed subject matter as described in more detail below. The encoded video data (404) (or encoded video bitstream (404)), depicted as a thin line to emphasize the lower data volume when compared to the stream of video pictures (402), can be stored on a streaming server (405) for future use. One or more streaming client subsystems, such as client subsystems (406) and (408) in FIG. 4 can access the streaming server (405) to retrieve copies (407) and (409) of the encoded video data (404). A client subsystem (406) can include a video decoder (410), for example, in an electronic device (430). The video decoder (410) decodes the incoming copy (407) of the encoded video data and creates an outgoing stream of video pictures (411) that can be rendered on a display (412) (e.g., display screen) or other rendering device (not depicted). In some streaming systems, the encoded video data (404), (407), and (409) (e.g., video bitstreams) can be encoded according to certain video coding/compression standards. Examples of those standards include ITU-T Recommendation H.265. In an example, a video coding standard under development is informally known as Versatile Video Coding (VVC). The disclosed subject matter may be used in the context of VVC.

It is noted that the electronic devices (420) and (430) can include other components (not shown). For example, the electronic device (420) can include a video decoder (not shown) and the electronic device (430) can include a video encoder (not shown) as well.

FIG. 5 shows a block diagram of a video decoder (510) according to an embodiment of the present disclosure. The video decoder (510) can be included in an electronic device (530). The electronic device (530) can include a receiver (531) (e.g., receiving circuitry). The video decoder (510) can be used in the place of the video decoder (410) in the FIG. 4 example.

The receiver (531) may receive one or more coded video sequences to be decoded by the video decoder (510); in the same or another embodiment, one coded video sequence at a time, where the decoding of each coded video sequence is independent from other coded video sequences. The coded video sequence may be received from a channel (501), which may be a hardware/software link to a storage device which stores the encoded video data. The receiver (531) may receive the encoded video data with other data, for example, coded audio data and/or ancillary data streams, that may be forwarded to their respective using entities (not depicted). The receiver (531) may separate the coded video sequence from the other data. To combat network jitter, a buffer memory (515) may be coupled in between the receiver (531) and an entropy decoder/parser (520) ("parser (520)" henceforth). In certain applications, the buffer memory (515) is part of the video decoder (510). In others, it can be outside of the video decoder (510) (not depicted). In still others, there can be a buffer memory (not depicted) outside of the video decoder (510), for example to combat network jitter,

and in addition another buffer memory (515) inside the video decoder (510), for example to handle playout timing. When the receiver (531) is receiving data from a store/forward device of sufficient bandwidth and controllability, or from an isosynchronous network, the buffer memory (515) may not be needed, or can be small. For use on best effort packet networks such as the Internet, the buffer memory (515) may be required, can be comparatively large and can be advantageously of adaptive size, and may at least partially be implemented in an operating system or similar elements (not depicted) outside of the video decoder (510).

The video decoder (510) may include the parser (520) to reconstruct symbols (521) from the coded video sequence. Categories of those symbols include information used to manage operation of the video decoder (510), and potentially information to control a rendering device such as a render device (512) (e.g., a display screen) that is not an integral part of the electronic device (530) but can be coupled to the electronic device (530), as was shown in FIG. 5. The control information for the rendering device(s) may be in the form of Supplemental Enhancement Information (SEI messages) or Video Usability Information (VUI) parameter set fragments (not depicted). The parser (520) may parse/entropy-decode the coded video sequence that is received. The coding of the coded video sequence can be in accordance with a video coding technology or standard, and can follow various principles, including variable length coding, Huffman coding, arithmetic coding with or without context sensitivity, and so forth. The parser (520) may extract from the coded video sequence, a set of subgroup parameters for at least one of the subgroups of pixels in the video decoder, based upon at least one parameter corresponding to the group. Subgroups can include Groups of Pictures (GOPs), pictures, tiles, slices, macroblocks, Coding Units (CUs), blocks, Transform Units (TUs), Prediction Units (PUs) and so forth. The parser (520) may also extract from the coded video sequence information such as transform coefficients, quantizer parameter values, motion vectors, and so forth.

The parser (520) may perform an entropy decoding/parsing operation on the video sequence received from the buffer memory (515), so as to create symbols (521).

Reconstruction of the symbols (521) can involve multiple different units depending on the type of the coded video picture or parts thereof (such as: inter and intra picture, inter and intra block), and other factors. Which units are involved, and how, can be controlled by the subgroup control information that was parsed from the coded video sequence by the parser (520). The flow of such subgroup control information between the parser (520) and the multiple units below is not depicted for clarity.

Beyond the functional blocks already mentioned, the video decoder (510) can be conceptually subdivided into a number of functional units as described below. In a practical implementation operating under commercial constraints, many of these units interact closely with each other and can, at least partly, be integrated into each other. However, for the purpose of describing the disclosed subject matter, the conceptual subdivision into the functional units below is appropriate.

A first unit is the scaler/inverse transform unit (551). The scaler/inverse transform unit (551) receives a quantized transform coefficient as well as control information, including which transform to use, block size, quantization factor, quantization scaling matrices, etc. as symbol(s) (521) from

the parser (520). The scaler/inverse transform unit (551) can output blocks comprising sample values, that can be input into aggregator (555).

In some cases, the output samples of the scaler/inverse transform (551) can pertain to an intra coded block; that is: a block that is not using predictive information from previously reconstructed pictures, but can use predictive information from previously reconstructed parts of the current picture. Such predictive information can be provided by an intra picture prediction unit (552). In some cases, the intra picture prediction unit (552) generates a block of the same size and shape of the block under reconstruction, using surrounding already reconstructed information fetched from the current picture buffer (558). The current picture buffer (558) buffers, for example, partly reconstructed current picture and/or fully reconstructed current picture. The aggregator (555), in some cases, adds, on a per sample basis, the prediction information the intra prediction unit (552) has generated to the output sample information as provided by the scaler/inverse transform unit (551).

In other cases, the output samples of the scaler/inverse transform unit (551) can pertain to an inter coded, and potentially motion compensated block. In such a case, a motion compensation prediction unit (553) can access reference picture memory (557) to fetch samples used for prediction. After motion compensating the fetched samples in accordance with the symbols (521) pertaining to the block, these samples can be added by the aggregator (555) to the output of the scaler/inverse transform unit (551) (in this case called the residual samples or residual signal) so as to generate output sample information. The addresses within the reference picture memory (557) from where the motion compensation prediction unit (553) fetches prediction samples can be controlled by motion vectors, available to the motion compensation prediction unit (553) in the form of symbols (521) that can have, for example X, Y, and reference picture components. Motion compensation also can include interpolation of sample values as fetched from the reference picture memory (557) when sub-sample exact motion vectors are in use, motion vector prediction mechanisms, and so forth.

The output samples of the aggregator (555) can be subject to various loop filtering techniques in the loop filter unit (556). Video compression technologies can include in-loop filter technologies that are controlled by parameters included in the coded video sequence (also referred to as coded video bitstream) and made available to the loop filter unit (556) as symbols (521) from the parser (520), but can also be responsive to meta-information obtained during the decoding of previous (in decoding order) parts of the coded picture or coded video sequence, as well as responsive to previously reconstructed and loop-filtered sample values.

The output of the loop filter unit (556) can be a sample stream that can be output to the render device (512) as well as stored in the reference picture memory (557) for use in future inter-picture prediction.

Certain coded pictures, once fully reconstructed, can be used as reference pictures for future prediction. For example, once a coded picture corresponding to a current picture is fully reconstructed and the coded picture has been identified as a reference picture (by, for example, the parser (520)), the current picture buffer (558) can become a part of the reference picture memory (557), and a fresh current picture buffer can be reallocated before commencing the reconstruction of the following coded picture.

The video decoder (510) may perform decoding operations according to a predetermined video compression tech-

nology in a standard, such as ITU-T Rec. H.265. The coded video sequence may conform to a syntax specified by the video compression technology or standard being used, in the sense that the coded video sequence adheres to both the syntax of the video compression technology or standard and the profiles as documented in the video compression technology or standard. Specifically, a profile can select certain tools as the only tools available for use under that profile from all the tools available in the video compression technology or standard. Also necessary for compliance can be that the complexity of the coded video sequence is within bounds as defined by the level of the video compression technology or standard. In some cases, levels restrict the maximum picture size, maximum frame rate, maximum reconstruction sample rate (measured in, for example megasamples per second), maximum reference picture size, and so on. Limits set by levels can, in some cases, be further restricted through Hypothetical Reference Decoder (HRD) specifications and metadata for HRD buffer management signaled in the coded video sequence.

In an embodiment, the receiver (531) may receive additional (redundant) data with the encoded video. The additional data may be included as part of the coded video sequence(s). The additional data may be used by the video decoder (510) to properly decode the data and/or to more accurately reconstruct the original video data. Additional data can be in the form of, for example, temporal, spatial, or signal noise ratio (SNR) enhancement layers, redundant slices, redundant pictures, forward error correction codes, and so on.

FIG. 6 shows a block diagram of a video encoder (603) according to an embodiment of the present disclosure. The video encoder (603) is included in an electronic device (620). The electronic device (620) includes a transmitter (640) (e.g., transmitting circuitry). The video encoder (603) can be used in the place of the video encoder (403) in the FIG. 4 example.

The video encoder (603) may receive video samples from a video source (601) (that is not part of the electronic device (620) in the FIG. 6 example) that may capture video image(s) to be coded by the video encoder (603). In another example, the video source (601) is a part of the electronic device (620).

The video source (601) may provide the source video sequence to be coded by the video encoder (603) in the form of a digital video sample stream that can be of any suitable bit depth (for example: 8 bit, 10 bit, 12 bit, . . .), any colorspace (for example, BT.601 Y CrCb, RGB, . . .), and any suitable sampling structure (for example Y CrCb 4:2:0, Y CrCb 4:4:4). In a media serving system, the video source (601) may be a storage device storing previously prepared video. In a videoconferencing system, the video source (601) may be a camera that captures local image information as a video sequence. Video data may be provided as a plurality of individual pictures that impart motion when viewed in sequence. The pictures themselves may be organized as a spatial array of pixels, wherein each pixel can comprise one or more samples depending on the sampling structure, color space, etc. in use. A person skilled in the art can readily understand the relationship between pixels and samples. The description below focuses on samples.

According to an embodiment, the video encoder (603) may code and compress the pictures of the source video sequence into a coded video sequence (643) in real time or under any other time constraints as required by the application. Enforcing appropriate coding speed is one function of a controller (650). In some embodiments, the controller

11

(650) controls other functional units as described below and is functionally coupled to the other functional units. The coupling is not depicted for clarity. Parameters set by the controller (650) can include rate control related parameters (picture skip, quantizer, lambda value of rate-distortion optimization techniques, . . .), picture size, group of pictures (GOP) layout, maximum motion vector search range, and so forth. The controller (650) can be configured to have other suitable functions that pertain to the video encoder (603) optimized for a certain system design.

In some embodiments, the video encoder (603) is configured to operate in a coding loop. As an oversimplified description, in an example, the coding loop can include a source coder (630) (e.g., responsible for creating symbols, such as a symbol stream, based on an input picture to be coded, and a reference picture(s)), and a (local) decoder (633) embedded in the video encoder (603). The decoder (633) reconstructs the symbols to create the sample data in a similar manner as a (remote) decoder also would create (as any compression between symbols and coded video bit-stream is lossless in the video compression technologies considered in the disclosed subject matter). The reconstructed sample stream (sample data) is input to the reference picture memory (634). As the decoding of a symbol stream leads to bit-exact results independent of decoder location (local or remote), the content in the reference picture memory (634) is also bit exact between the local encoder and remote encoder. In other words, the prediction part of an encoder “sees” as reference picture samples exactly the same sample values as a decoder would “see” when using prediction during decoding. This fundamental principle of reference picture synchronicity (and resulting drift, if synchronicity cannot be maintained, for example because of channel errors) is used in some related arts as well.

The operation of the “local” decoder (633) can be the same as of a “remote” decoder, such as the video decoder (510), which has already been described in detail above in conjunction with FIG. 5. Briefly referring also to FIG. 5, however, as symbols are available and encoding/decoding of symbols to a coded video sequence by an entropy coder (645) and the parser (520) can be lossless, the entropy decoding parts of the video decoder (510), including the buffer memory (515), and parser (520) may not be fully implemented in the local decoder (633).

An observation that can be made at this point is that any decoder technology except the parsing/entropy decoding that is present in a decoder also necessarily needs to be present, in substantially identical functional form, in a corresponding encoder. For this reason, the disclosed subject matter focuses on decoder operation. The description of encoder technologies can be abbreviated as they are the inverse of the comprehensively described decoder technologies. Only in certain areas a more detail description is required and provided below.

During operation, in some examples, the source coder (630) may perform motion compensated predictive coding, which codes an input picture predictively with reference to one or more previously coded picture from the video sequence that were designated as “reference pictures.” In this manner, the coding engine (632) codes differences between pixel blocks of an input picture and pixel blocks of reference picture(s) that may be selected as prediction reference(s) to the input picture.

The local video decoder (633) may decode coded video data of pictures that may be designated as reference pictures, based on symbols created by the source coder (630). Opera-

12

tions of the coding engine (632) may advantageously be lossy processes. When the coded video data may be decoded at a video decoder (not shown in FIG. 6), the reconstructed video sequence typically may be a replica of the source video sequence with some errors. The local video decoder (633) replicates decoding processes that may be performed by the video decoder on reference pictures and may cause reconstructed reference pictures to be stored in the reference picture cache (634). In this manner, the video encoder (603) may store copies of reconstructed reference pictures locally that have common content as the reconstructed reference pictures that will be obtained by a far-end video decoder (absent transmission errors).

The predictor (635) may perform prediction searches for the coding engine (632). That is, for a new picture to be coded, the predictor (635) may search the reference picture memory (634) for sample data (as candidate reference pixel blocks) or certain metadata such as reference picture motion vectors, block shapes, and so on, that may serve as an appropriate prediction reference for the new pictures. The predictor (635) may operate on a sample block-by-pixel block basis to find appropriate prediction references. In some cases, as determined by search results obtained by the predictor (635), an input picture may have prediction references drawn from multiple reference pictures stored in the reference picture memory (634).

The controller (650) may manage coding operations of the source coder (630), including, for example, setting of parameters and subgroup parameters used for encoding the video data.

Output of all aforementioned functional units may be subjected to entropy coding in the entropy coder (645). The entropy coder (645) translates the symbols as generated by the various functional units into a coded video sequence, by lossless compressing the symbols according to technologies such as Huffman coding, variable length coding, arithmetic coding, and so forth.

The transmitter (640) may buffer the coded video sequence(s) as created by the entropy coder (645) to prepare for transmission via a communication channel (660), which may be a hardware/software link to a storage device which would store the encoded video data. The transmitter (640) may merge coded video data from the video coder (603) with other data to be transmitted, for example, coded audio data and/or ancillary data streams (sources not shown).

The controller (650) may manage operation of the video encoder (603). During coding, the controller (650) may assign to each coded picture a certain coded picture type, which may affect the coding techniques that may be applied to the respective picture. For example, pictures often may be assigned as one of the following picture types:

An Intra Picture (I picture) may be one that may be coded and decoded without using any other picture in the sequence as a source of prediction. Some video codecs allow for different types of intra pictures, including, for example Independent Decoder Refresh (“IDR”) Pictures. A person skilled in the art is aware of those variants of I pictures and their respective applications and features.

A predictive picture (P picture) may be one that may be coded and decoded using intra prediction or inter prediction using at most one motion vector and reference index to predict the sample values of each block.

A bi-directionally predictive picture (B Picture) may be one that may be coded and decoded using intra prediction or inter prediction using at most two motion vectors and reference indices to predict the sample values of each block. Similarly, multiple-predictive pictures can use more than

two reference pictures and associated metadata for the reconstruction of a single block.

Source pictures commonly may be subdivided spatially into a plurality of sample blocks (for example, blocks of 4×4, 8×8, 4×8, or 16×16 samples each) and coded on a block-by-block basis. Blocks may be coded predictively with reference to other (already coded) blocks as determined by the coding assignment applied to the blocks' respective pictures. For example, blocks of I pictures may be coded non-predictively or they may be coded predictively with reference to already coded blocks of the same picture (spatial prediction or intra prediction). Pixel blocks of P pictures may be coded predictively, via spatial prediction or via temporal prediction with reference to one previously coded reference picture. Blocks of B pictures may be coded predictively, via spatial prediction or via temporal prediction with reference to one or two previously coded reference pictures.

The video encoder (603) may perform coding operations according to a predetermined video coding technology or standard, such as ITU-T Rec. H.265. In its operation, the video encoder (603) may perform various compression operations, including predictive coding operations that exploit temporal and spatial redundancies in the input video sequence. The coded video data, therefore, may conform to a syntax specified by the video coding technology or standard being used.

In an embodiment, the transmitter (640) may transmit additional data with the encoded video. The source coder (630) may include such data as part of the coded video sequence. Additional data may comprise temporal/spatial/SNR enhancement layers, other forms of redundant data such as redundant pictures and slices, SEI messages, VUI parameter set fragments, and so on.

A video may be captured as a plurality of source pictures (video pictures) in a temporal sequence. Intra-picture prediction (often abbreviated to intra prediction) makes use of spatial correlation in a given picture, and inter-picture prediction makes use of the (temporal or other) correlation between the pictures. In an example, a specific picture under encoding/decoding, which is referred to as a current picture, is partitioned into blocks. When a block in the current picture is similar to a reference block in a previously coded and still buffered reference picture in the video, the block in the current picture can be coded by a vector that is referred to as a motion vector. The motion vector points to the reference block in the reference picture, and can have a third dimension identifying the reference picture, in case multiple reference pictures are in use.

In some embodiments, a bi-prediction technique can be used in the inter-picture prediction. According to the bi-prediction technique, two reference pictures, such as a first reference picture and a second reference picture that are both prior in decoding order to the current picture in the video (but may be in the past and future, respectively, in display order) are used. A block in the current picture can be coded by a first motion vector that points to a first reference block in the first reference picture, and a second motion vector that points to a second reference block in the second reference picture. The block can be predicted by a combination of the first reference block and the second reference block.

Further, a merge mode technique can be used in the inter-picture prediction to improve coding efficiency.

According to some embodiments of the disclosure, predictions, such as inter-picture predictions and intra-picture predictions are performed in the unit of blocks. For example, according to the HEVC standard, a picture in a sequence of

video pictures is partitioned into coding tree units (CTU) for compression, the CTUs in a picture have the same size, such as 64×64 pixels, 32×32 pixels, or 16×16 pixels. In general, a CTU includes three coding tree blocks (CTBs), which are one luma CTB and two chroma CTBs. Each CTU can be recursively quadtree split into one or multiple coding units (CUs). For example, a CTU of 64×64 pixels can be split into one CU of 64×64 pixels, or 4 CUs of 32×32 pixels, or 16 CUs of 16×16 pixels. In an example, each CU is analyzed to determine a prediction type for the CU, such as an inter prediction type or an intra prediction type. The CU is split into one or more prediction units (PUs) depending on the temporal and/or spatial predictability. Generally, each PU includes a luma prediction block (PB), and two chroma PBs. In an embodiment, a prediction operation in coding (encoding/decoding) is performed in the unit of a prediction block. Using a luma prediction block as an example of a prediction block, the prediction block includes a matrix of values (e.g., luma values) for pixels, such as 8×8 pixels, 16×16 pixels, 8×16 pixels, 16×8 pixels, and the like.

FIG. 7 shows a diagram of a video encoder (703) according to another embodiment of the disclosure. The video encoder (703) is configured to receive a processing block (e.g., a prediction block) of sample values within a current video picture in a sequence of video pictures, and encode the processing block into a coded picture that is part of a coded video sequence. In an example, the video encoder (703) is used in the place of the video encoder (403) in the FIG. 4 example.

In an HEVC example, the video encoder (703) receives a matrix of sample values for a processing block, such as a prediction block of 8×8 samples, and the like. The video encoder (703) determines whether the processing block is best coded using intra mode, inter mode, or bi-prediction mode using, for example, rate-distortion optimization. When the processing block is to be coded in intra mode, the video encoder (703) may use an intra prediction technique to encode the processing block into the coded picture; and when the processing block is to be coded in inter mode or bi-prediction mode, the video encoder (703) may use an inter prediction or bi-prediction technique, respectively, to encode the processing block into the coded picture. In certain video coding technologies, merge mode can be an inter picture prediction submode where the motion vector is derived from one or more motion vector predictors without the benefit of a coded motion vector component outside the predictors. In certain other video coding technologies, a motion vector component applicable to the subject block may be present. In an example, the video encoder (703) includes other components, such as a mode decision module (not shown) to determine the mode of the processing blocks.

In the FIG. 7 example, the video encoder (703) includes the inter encoder (730), an intra encoder (722), a residue calculator (723), a switch (726), a residue encoder (724), a general controller (721), and an entropy encoder (725) coupled together as shown in FIG. 7.

The inter encoder (730) is configured to receive the samples of the current block (e.g., a processing block), compare the block to one or more reference blocks in reference pictures (e.g., blocks in previous pictures and later pictures), generate inter prediction information (e.g., description of redundant information according to inter encoding technique, motion vectors, merge mode information), and calculate inter prediction results (e.g., predicted block) based on the inter prediction information using any

15

suitable technique. In some examples, the reference pictures are decoded reference pictures that are decoded based on the encoded video information.

The intra encoder (722) is configured to receive the samples of the current block (e.g., a processing block), in some cases compare the block to blocks already coded in the same picture, generate quantized coefficients after transform, and in some cases also intra prediction information (e.g., an intra prediction direction information according to one or more intra encoding techniques). In an example, the intra encoder (722) also calculates intra prediction results (e.g., predicted block) based on the intra prediction information and reference blocks in the same picture.

The general controller (721) is configured to determine general control data and control other components of the video encoder (703) based on the general control data. In an example, the general controller (721) determines the mode of the block, and provides a control signal to the switch (726) based on the mode. For example, when the mode is the intra mode, the general controller (721) controls the switch (726) to select the intra mode result for use by the residue calculator (723), and controls the entropy encoder (725) to select the intra prediction information and include the intra prediction information in the bitstream; and when the mode is the inter mode, the general controller (721) controls the switch (726) to select the inter prediction result for use by the residue calculator (723), and controls the entropy encoder (725) to select the inter prediction information and include the inter prediction information in the bitstream.

The residue calculator (723) is configured to calculate a difference (residue data) between the received block and prediction results selected from the intra encoder (722) or the inter encoder (730). The residue encoder (724) is configured to operate based on the residue data to encode the residue data to generate the transform coefficients. In an example, the residue encoder (724) is configured to convert the residue data from a spatial domain to a frequency domain, and generate the transform coefficients. The transform coefficients are then subject to quantization processing to obtain quantized transform coefficients. In various embodiments, the video encoder (703) also includes a residue decoder (728). The residue decoder (728) is configured to perform inverse-transform, and generate the decoded residue data. The decoded residue data can be suitably used by the intra encoder (722) and the inter encoder (730). For example, the inter encoder (730) can generate decoded blocks based on the decoded residue data and inter prediction information, and the intra encoder (722) can generate decoded blocks based on the decoded residue data and the intra prediction information. The decoded blocks are suitably processed to generate decoded pictures and the decoded pictures can be buffered in a memory circuit (not shown) and used as reference pictures in some examples.

The entropy encoder (725) is configured to format the bitstream to include the encoded block. The entropy encoder (725) is configured to include various information according to a suitable standard, such as the HEVC standard. In an example, the entropy encoder (725) is configured to include the general control data, the selected prediction information (e.g., intra prediction information or inter prediction information), the residue information, and other suitable information in the bitstream. Note that, according to the disclosed subject matter, when coding a block in the merge submode of either inter mode or bi-prediction mode, there is no residue information.

FIG. 8 shows a diagram of a video decoder (810) according to another embodiment of the disclosure. The video

16

decoder (810) is configured to receive coded pictures that are part of a coded video sequence, and decode the coded pictures to generate reconstructed pictures. In an example, the video decoder (810) is used in the place of the video decoder (410) in the FIG. 4 example.

In the FIG. 8 example, the video decoder (810) includes an entropy decoder (871), an inter decoder (880), a residue decoder (873), a reconstruction module (874), and an intra decoder (872) coupled together as shown in FIG. 8.

The entropy decoder (871) can be configured to reconstruct, from the coded picture, certain symbols that represent the syntax elements of which the coded picture is made up. Such symbols can include, for example, the mode in which a block is coded (such as, for example, intra mode, inter mode, bi-predicted mode, the latter two in merge submode or another submode), prediction information (such as, for example, intra prediction information or inter prediction information) that can identify certain sample or metadata that is used for prediction by the intra decoder (872) or the inter decoder (880), respectively, residual information in the form of, for example, quantized transform coefficients, and the like. In an example, when the prediction mode is inter or bi-predicted mode, the inter prediction information is provided to the inter decoder (880); and when the prediction type is the intra prediction type, the intra prediction information is provided to the intra decoder (872). The residual information can be subject to inverse quantization and is provided to the residue decoder (873).

The inter decoder (880) is configured to receive the inter prediction information, and generate inter prediction results based on the inter prediction information.

The intra decoder (872) is configured to receive the intra prediction information, and generate prediction results based on the intra prediction information.

The residue decoder (873) is configured to perform inverse quantization to extract de-quantized transform coefficients, and process the de-quantized transform coefficients to convert the residual from the frequency domain to the spatial domain. The residue decoder (873) may also require certain control information (to include the Quantizer Parameter (QP)), and that information may be provided by the entropy decoder (871) (data path not depicted as this may be low volume control information only).

The reconstruction module (874) is configured to combine, in the spatial domain, the residual as output by the residue decoder (873) and the prediction results (as output by the inter or intra prediction modules as the case may be) to form a reconstructed block, that may be part of the reconstructed picture, which in turn may be part of the reconstructed video. It is noted that other suitable operations, such as a deblocking operation and the like, can be performed to improve the visual quality.

It is noted that the video encoders (403), (603), and (703), and the video decoders (410), (510), and (810) can be implemented using any suitable technique. In an embodiment, the video encoders (403), (603), and (703), and the video decoders (410), (510), and (810) can be implemented using one or more integrated circuits. In another embodiment, the video encoders (403), (603), and (603), and the video decoders (410), (510), and (810) can be implemented using one or more processors that execute software instructions.

Aspects of the disclosure provide techniques for context-based adaptive binary arithmetic coding (CABAC) initialization. In some embodiments, context initialization for CABAC is based on inheriting information from previously coded (encoded/decoded) frames.

CABAC is a coding technique used in entropy coding. Generally, the encoding process of CABAC includes a binarization step, a context modeling step and an arithmetic coding step.

In the binarization step for the CABAC based encoding process, a syntax element of nonbinary value can be mapped to a binary sequence, also referred to as a bin string. The binarization step can be bypassed when the syntax element is provided of a value in binary form (e.g., a binary sequence).

In the context modeling step for the CABAC based encoding process, a probability model is determined depending on previously encoded syntax elements. In some examples, a probability model (also referred to as context in CABAC) can be represented by a probability state (also referred to as context state in some examples) and a most probable symbol (MPS) value. The probability state is associated with a probability value and can implicitly represent that the probability of a particular symbol (e.g., a bin) being the Least Probable Symbol (LPS) is equal to the probability value. A symbol can be an LPS or an MPS. For binary symbol, the MPS and the LPS can be 0 or 1. For example, if the LPS is 1, the MPS is 0; and if the LPS is 0, the MPS is 1. The probability value is estimated for the corresponding context and can be used to entropy code the symbol using the arithmetic coder.

The arithmetic coding step of the CABAC based encoding process is based on the principle of recursive interval subdivision according to the probability model. In some examples, the arithmetic coding step is handled by a state machine with a range parameter and a low parameter. The state machine can change values of the range parameter and the low parameter based on the contexts (probability models) and a sequence of bins to code. The value of the range parameter indicates a size of a current range that the coded value (of bins) falls into, and the value of the low parameter indicates the lower boundary of the current range. In an example, according to a probability state (e.g., in association with a probability value), a current range (CurrRange) is divided into a first subrange (MpsRange) (also referred to as MPS range of the current state) and a second subrange (LpsRange) (also referred to as LPS range of the current state). In an example, the second subrange can be calculated by a multiplication, such as using Eq. (1)

$$\text{LpsRange} = \text{CurrRange} \times \rho \quad \text{Eq. (1)}$$

where ρ is the probability value that the current bin is the LPS. The probability that the current bin is MPS can be calculated by $(1-\rho)$. The first subrange can be calculated by Eq. (2):

$$\text{MpsRange} = \text{CurrRange} - \text{LpsRange} \quad \text{Eq. (2)}$$

In an example, when the current bin is MPS, the value of the low parameter is kept, and the value of the range parameter is updated to MpsRange; and when the current bin is LPS, the value of the low parameter is updated to $(\text{low} + \text{MpsRange})$, and the value of the range parameter is updated to LpsRange. Then, the encoding process of CABAC can continue to a next bin in the sequence of bins.

In some examples (e.g., HEVC), the value of the range parameter is expressed with 9 bits and the value of the low parameter is expressed with 10 bits. Further, a renormalization process can be performed to maintain the range and low values at sufficient precision. For example, the renormalization can be performed whenever the value of the range parameter is less than 256. Therefore, the range parameter is equal or larger than 256 after renormalization.

In some examples (e.g., HEVC), 64 possible probability values for the LPS can be used and each MPS can be 0 or 1. In an example, the probability models can be stored as 7-bit entries that correspond to 64 probability values (64 probability states) and 2 possible values for MPS (0 or 1). In each of the 7-bit entries, 6 bits may be allocated for representing the probability state, and 1 bit may be allocated for the MPS.

In some examples, to reduce the computation of deriving LPS ranges (e.g., multiplications in Eq. (1)), results for all cases are pre-calculated, quantized and stored as approximations in a look-up table. Therefore, the LPS range can be obtained by using a table lookup without any multiplication operations. Avoiding multiplication can be important for some devices or applications, to reduce computation and latency.

FIG. 9 shows a table (900) of quantized values of LPS ranges in some examples. In some examples, the range parameter is expressed with 9 bits, and the values of the range parameter are equal or greater than 256. The ranges can be divided into four segments, such as referred to as segment 0, segment 1, segment 2 and segment 3. The segment 0 includes 64 ranges with values from 256 to 319; the segment 1 includes 64 ranges with values from 320 to 383; the segment 2 includes 64 ranges with values from 384 to 447; the segment 3 includes 64 ranges with values from 448 to 511. The index of the segments can be derived using $(\text{range} \gg 6) \& 3$ in an example, where range denotes to the range parameter expressed with 9 bits. In some examples, the range parameter is expressed with 9 bits, then the quantized values of the LPS range can be expressed using 8 bits, thus values in the table (900) can be expressed using 8 bits.

In the FIG. 9 example, the table (900) includes 4 columns respectively corresponding the 4 segments of ranges and includes 64 rows respectively corresponding to 64 probability states (associated with respective probability values). A value stored at an entry with a row index and a column index is a quantized value for a LPS range in association with a probability state (corresponding to the row index) and a range segment (corresponding to the column index). In the FIG. 9 example, LPS ranges for a probability state are quantized into four values (i.e., one value for each segment).

In some examples, a CABAC engine for performing CABAC based encoding or decoding (e.g., in a video encoder or in a video decoder), can include the table (900) that stores 64×4 8-bit values to approximate the calculations in Eq. (1). The table (900) can be used for performing a table-based probability transition process between 64 different probability states. For example, for a current range, a probability state is determined. The current range is used to determine a range segment that indicates a column index in the table (900). The probability state is used to determine a row index in the table (900). Then, a table lookup is performed to the table (900) to obtain a value stored in an entry having the row index and the column index, and the value is the approximated LPS range.

In some examples (e.g., VVC), the probability value of a probability state is linearly expressed by a probability index (denoted by pStateldx), and calculation can be done with equations without LUT operation. To improve the accuracy of probability estimation, a multi-hypothesis probability update model can be applied. For example, the pStateldx used in the interval subdivision in the binary arithmetic coder is a combination of two probabilities pStateldx0 and pStateldx1 (in the case of two hypotheses). The two probabilities are associated with respective context models and

19

are updated independently with different adaptation rates. The adaptation rates of pStateIdx0 and pStateIdx1 for the respective context models can be pre-trained based on the statistics of the associated bins. In some examples, the probability estimate pStateIdx is an average of the estimates from the two hypotheses.

FIG. 10 shows a flowchart outlining a process (1000) for decoding a single binary decision in some examples (e.g., VVC). In some examples, context table (e.g., denoted by a variable ctxTable), and context index (e.g., denoted by ctxIdx), can be provided to the process (1000) to obtain the probability state information, such as two probabilities pStateIdx0 and pStateIdx1. The process (1000) can operate on variables ivlCurrRange (indicative of the current range), and ivlOffset (indicative of the lower boundary of the current range) based on the probability state information, such as the two probabilities pStateIdx0 and pStateIdx1, and can output the decoded value binVal, and update the variables ivlCurrRange and ivlOffset, and the two probabilities pStateIdx0 and pStateIdx1. It is noted that, in some examples, the variables ivlCurrRange and ivlOffset are defined in certain way that ivlOffset is greater than or equal to ivlCurrRange. The process (1000) starts at (S1001), and proceeds to (S1010).

At (S1010), the value of the LPS range (denoted by a variable ivlLpsRange) is derived. In an example, given the current value of ivlCurrRange, a variable qRangeIdx is derived as Eq. (3):

$$qRangeIdx = ivlCurrRange \gg 5 \quad \text{Eq. (3)}$$

Then, given qRangeIdx, pStateIdx0 and pStateIdx1 (associated with ctxTable and ctxIdx), the value of MPS (denoted by valMps) and the value of LPS range (denoted by ivlLpsRange) are derived, for example according to Eq. (4), Eq. (5) and Eq. (6):

$$pState = pStateIdx1 + 16 \times pStateIdx0 \quad \text{Eq. (4)}$$

$$valMps = pState \gg 14 \quad \text{Eq. (5)}$$

$$ivlLpsRange = (qRangeIdx \times ((valMps \gg 32767 - pState) \gg 9) \gg 1) + 4 \quad \text{Eq. (6)}$$

Further, the bin is assumed to be MPS, then the variable ivlCurrRange is set equal to (ivlCurrRange - ivlLpsRange).

At (S1020), if the variable ivlOffset is greater than or equal to ivlCurrRange, the assumption (the bin is MPS) is not true, and the process proceeds to (S1030); otherwise, the process proceeds to (S1040).

At (S1030), the bin is LPS and thus the bin value (denoted by the variable binVal) is set equal to (1 - valMps), the lower boundary of the current range and the current range are updated accordingly. In an example, the variable ivlOffset is decremented by ivlCurrRange, and ivlCurrRange is set equal to ivlLpsRange.

At (S1040), the bin is MPS, and the variable binVal is set equal to valMps.

At (S1050), state transition is performed. The probability state information, such as the two probabilities pStateIdx0 and pStateIdx1, is updated based on the decoded value binVal.

In some examples, two adaptation variables shift0 and shift1 are derived from the shiftIdx value that is associated with ctxTable and ctxIdx, such as shown by Eq. (7) and Eq. (8):

$$shift0 = (shiftIdx \gg 2) + 2 \quad \text{Eq. (7)}$$

$$shift1 = (shiftIdx \& 3) + 3 + shift0 \quad \text{Eq. (8)}$$

20

The probability state information, such as the two probabilities pStateIdx0 and pStateIdx1, is updated according to the decoded value binVal, such as shown by Eq. (9) and Eq. (10):

$$pStateIdx0 = pStateIdx0 - (pStateIdx0 \gg shift0) + (1023 \times binVal \gg shift0) \quad \text{Eq. (9)}$$

$$pStateIdx1 = pStateIdx1 - (pStateIdx1 \gg shift1) + (16383 \times binVal \gg shift1) \quad \text{Eq. (10)}$$

At (S1060), a renormalization step can be performed, for example, when the current range ivlCurrRange is less than 256. Then, the process (1000) proceeds to (S1099) and terminates.

According to an aspect of the disclosure, the probability state can be initialized at the beginning of each slice. In some examples (e.g., HEVC and VVC), CABAC engine can have a QP dependent initialization process invoked at the beginning of each slice. In an example, given the initial value of luma QP for a slice, the initial probability state of a context model, denoted as preCtxState, is derived according to Eq. (11)-Eq. (15):

$$slopeIdx = initValue \gg 3 \quad \text{Eq. (11)}$$

$$offsetIdx = initValue \& 7 \quad \text{Eq. (12)}$$

$$m = slopeIdx - 4 \quad \text{Eq. (13)}$$

$$n = (offsetIdx \times 18) + 1 \quad \text{Eq. (14)}$$

$$preCtxState = clip3(1, 127, ((m \times (Clip3(0, 63, SliceQpy) - 16)) \gg 1) + n) \quad \text{Eq. (15)}$$

where the variable initValue is a 6-bit variable that is obtained from tables associated with the variables ctxTable and ctxIdx, SliceQpy denotes the slice QP value, slopeIdx and offsetIdx are restricted to 3 bits. The probability state preCtxState represents the probability in the linear domain directly. It is noted that Clip3(x, y, z) is a function that clips z in the range of [x, y].

It is noted that, in some examples, proper shifting operations can be performed on preCtxState to generate inputs to arithmetic coding engine (for performing arithmetic coding step). In some examples, mapping from logarithmic to linear domain as well as the table (900) is not needed. In some examples, the two probabilities pStateIdx0 and pStateIdx1 can be derived from the initialization using shifting operations, such as according to Eq. (16) and Eq. (17):

$$pStateIdx0 = preCtxState \ll 3 \quad \text{Eq. (16)}$$

$$pStateIdx1 = preCtxState \ll 7 \quad \text{Eq. (17)}$$

According to some aspects of the disclosure, an inherited context initialization method can improve CABAC efficiency. For example, context models in CABAC can be adapted to content of a first frame after coding (encoding/decoding) of the first frame. Using the context models on a second frame that has similar content to the first frame can achieve better coding efficiency for coding the second frame. In some related examples for inherited context initialization, the context states of context models in an inter slice (i.e., with a slice type of B or P) that contains the last CTU in a picture can be stored. The stored context states can be used to initialize the context models in a next inter slice that has the same slice type, QP value and temporal layer ID (e.g., similar content). In some examples, for each inter slice, a control flag is signaled to indicate a context initialization scheme that is used for the inter slice. For example, the control flag being equal to zero can indicate that the context

models of the inter slice are initialized using one of the existing context initialization tables (e.g., the context initialization table can be indicated by a slice header level flag `sh_cabac_init_flag`); and the control flag being equal to one can indicate that the context models of the inter slice are initialized by inheriting the stored context states from the previously coded picture.

According to an aspect of the disclosure, the related examples can improve coding efficiency, but can break frame level parallelism such as frame parallel processing (FPP) in both encoder and decoder because the related examples introduce dependency between frames at the same temporal level (e.g., same temporal layer ID). In an example, frame level parallelism is used to allow processing multiple frames, such as a first frame and a second frame, at the same time. However, when context models for coding the second frame are initialized based the context states of context models of an inter slice that includes the last CTU of the first frame, then coding of the second frame needs to wait until the inter slice that includes the last CTU of the first frame is coded. Then, frame level parallelism cannot be used to process the first frame and the second frame at the same time.

According to some aspects of the disclosure, at an encoder side, the encoder can select a first portion of a video sequence as a source for inherited context initialization for coding a second portion of the video sequence. The first portion can be selected to avoid breaking the frame level parallelism. The encoder can include a signal in the coded bitstream to indicate the first portion as the source for inherited context initialization for coding the second portion. Then, at a decoder side, the decoder can decode the signal that indicates the source for inherited context initialization, and can perform inherited context initialization accordingly.

According to some aspects of the disclosure, a signal that indicates from which picture/slice that the CABAC states (including but not limited to contexts and window sizes associated with the adaptation rates) are inherited for the current picture/slice can be included in the bitstream, such as at picture level, picture header level or slice level. In an example, a signal that indicates from which picture that the CABAC states are inherited for the current picture is included in the bitstream, such as in a picture parameter set, in a picture header and the like. In another example, a signal that indicates from which slice that the CABAC states are inherited for the current slice is included in the bitstream, such as in a slice header.

In some embodiments, the encoder can select a coded portion that is a reference for coding the current portion, thus no extra dependency is introduced by the CABAC state inheritance. In an example, the encoder can select a coded picture that is a reference picture for coding the current picture (P or B type), and can include the signal in the bitstream to indicate the coded picture as the source for inherited context initialization. In another example, the encoder can select a coded slice that is a reference slice for coding the current slice (P or B type), and can include the signal in the bitstream to indicate the coded slice as the source for inherited context initialization.

In some embodiments, the encoder can select more than one portion as the source for the inherited context initialization, and can signal the more than one portion in the bitstream. The context models for the inherited context initialization can be determined based on a weighted average of the context models associated with the more than one portion. The weighted average can be calculated based on weighting factors respectively associated with the more than

one portion. The weighting factors can be predefined and known to the encoder and the decoder in an example. In another example, the weighting factors are signaled in the bitstream.

In an embodiment, the encoder can select a plurality of pictures as the source for the inherited context initialization, and can signal the plurality of pictures in the bitstream. The context models for the inherited context initialization can be determined based on a weighted average of the context models associated with the plurality of pictures. The weighted average can be calculated based on weighting factors respectively associated with the plurality of pictures. The weighting factors can be predefined and known to the encoder and the decoder in an example. In another example, the weighting factors are signaled in the bitstream.

In another embodiment, the encoder can select a plurality of slices as the source for the inherited context initialization, and can signal the plurality of slices in the bitstream. The context models for the inherited context initialization can be determined based on a weighted average of the context models associated with the plurality of slices. The weighted average can be calculated based on weighting factors respectively associated with the plurality of slices. The weighting factors can be predefined and known to the encoder and the decoder in an example. In another example, the weighting factors are signaled in the bitstream.

In some examples, a flag is signaled at slice/picture level (e.g., current picture, current slice) to indicate whether the CABAC state inheritance is allowed. If the flag indicates that the CABAC state inheritance is allowed, weighting factors can be further signaled for each reference (e.g., reference picture, reference slice). Weighting factor being zero indicates that a reference picture/slice is not used as a source picture for the inheritance. In an example, a weighting factor is not signaled for a reference (picture or slice), then the weighting factor for the reference (picture or slice) can be set to zero. In another example, a default weighting factor may be signaled in bitstreams. When no weighting factor is signaled for a reference (picture or slice), the default weighting factor is used for the reference (picture or slice).

In some examples, an indicator is signaled at a group of pictures (GOP) level to indicate which picture is allowed to be used as source of CABAC state inheritance. Then, the encoder and/or decoder can prioritize those pictures (that are allowed to be used as source of CABAC state inheritance) in parallel processing. For example, the encoder/decoder can arrange those pictures to be processed before other pictures. In an example, the indicator may be included in the reference picture list structure syntax, and the indicator indicates one or more reference pictures that are allowed to be used as source of CABAC state inheritance.

FIG. 11 shows a flow chart outlining a process (1100) according to an embodiment of the disclosure. The process (1100) can be used in a video encoder. In various embodiments, the process (1100) is executed by processing circuitry, such as the processing circuitry in the terminal devices (310), (320), (330) and (340), the processing circuitry that performs functions of the video encoder (403), the processing circuitry that performs functions of the video encoder (603), the processing circuitry that performs functions of the video encoder (703), and the like. In some embodiments, the process (1100) is implemented in software instructions, thus when the processing circuitry executes the software instructions, the processing circuitry performs the process (1100). The process starts at (S11301) and proceeds to (S1110).

At (S1110), the encoder determines a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for encoding a current portion of a video sequence.

At (S1120), the encoder determines an encoded portion in the video sequence for a context inheriting by the current portion according to a frame level parallelism for coding the video sequence. The context inheriting by the current portion to the encoded portion allows the frame level parallelism for coding the video sequence, and does not interfere or break the frame level parallelism. In an example, according to a frame level parallelism, a first frame and a second frame can be coded (encoded or decoded) in parallel, then context inheriting does not introduce dependency between the first frame and the second frame, and allows the first frame and the second frame to be coded in parallel.

At (S1130), the encoder encodes a signal (e.g., syntax value) indicative of the encoded portion for the context inheriting. In an example, the signal indicates an encoded picture for the context inheriting. In another example, the signal indicates an encoded slice for the context inheriting. The signal can be encoded in at least one of a picture parameter set (PPS), a picture header, and a slice header.

In some examples, the encoded portion is a reference that is used for the encoding of the current portion. Thus, the context inheriting does not introduce additional dependency that may break the frame level parallelism.

In some examples, the encoder encodes a first signal and a second signal in a bitstream that carries the video sequence, the first signal indicates the use of the inherited context initialization in the CABAC for encoding the current portion, and the second signal indicates the encoded portion in the video sequence for the context inheriting.

At (S1140), the encoder initializes, context models for the CABAC based on stored context states associated with the encoded portion.

In some embodiments, the signal indicates at least a first encoded portion and a second encoded portion. Then, the context models for the CABAC can be initialized according to a weighted average of probability values indicated by at least first context states associated with the first encoded portion and second context states associated with the second encoded portion. For example, to initialize a context model, a first probability is determined based on a first stored context state associated with the first encoded portion, and a second probability is determined based on a second stored context state associated with the second encoded portion. A weighted average is then calculated based on the first probability and the second probability, and the context model is initialized according to the weighted average.

In some examples, weighting factors for calculating the weighted average can be encoded in the bitstream. In an example, if no weighting factor is signaled for an encoded portion, zero weighting factor is used. In another example, if no weighting factor is signaled for an encoded portion, a default weighting factor can be used. The default weighting factor can be signaled in the bitstream or can be predefined and known at the encoder side and the decoder side.

At (S1150), the encoder encodes, the current portion based on the context models that are initialized based on the stored context states associated with the encoded portion.

In some examples, the encoder can encode an indicator at a group of pictures (GOP) level. The indicator can indicate one or more pictures that are allowed for the context inheriting. The indicator can assist the decoder side to prioritize the decoding for the one or more pictures. In an example, the indicator is encoded in a reference picture list

structure syntax to indicate one or more reference pictures in the reference picture list structure syntax.

Then, the process proceeds to (S1199) and terminates.

The process (1100) can be suitably adapted. Step(s) in the process (1100) can be modified and/or omitted. Additional step(s) can be added. Any suitable order of implementation can be used.

FIG. 12 shows a flow chart outlining a process (1200) according to an embodiment of the disclosure. The process (1200) can be used in a video decoder. In various embodiments, the process (1200) is executed by processing circuitry, such as the processing circuitry in the terminal devices (310), (320), (330) and (340), the processing circuitry that performs functions of the video decoder (410), the processing circuitry that performs functions of the video decoder (510), and the like. In some embodiments, the process (1200) is implemented in software instructions, thus when the processing circuitry executes the software instructions, the processing circuitry performs the process (1200). The process starts at (S1201) and proceeds to (S1210).

At (S1210), the decoder determines a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for decoding a current portion of a video sequence that is carried by a bitstream. In an example, the decoder decodes, from the bitstream, a control signal that indicates the use of the inherited context initialization in the CABAC for decoding the current portion of the video sequence.

At (S1220), the decoder decodes, from the bitstream, a signal (e.g., syntax value) that indicates a decoded portion in the video sequence for a context inheriting.

In an example, the signal indicates a decoded picture for the context inheriting. In another example, the signal indicates a decoded slice for the context inheriting. The signal can be decoded from at least one of a picture parameter set (PPS), a picture header, and a slice header.

In some examples, the decoded portion is a reference (e.g., a reference picture, a reference slice) that is used for the decoding of the current portion, and thus no additional dependency is introduced by the context inheriting.

At (S1230), the decoder initializes, context models for the CABAC based on stored context states associated with the decoded portion.

In some examples, the signal can indicate at least a first decoded portion and a second decoded portion. The context models for the CABAC can be initialized according to a weighted average of probability values indicated by at least first context states associated with the first decoded portion and second context states associated with the second decoded portion. For example, to initialize a context model, a first probability is determined based on a first stored context state associated with the first decoded portion, and a second probability is determined based on a second stored context state associated with the second decoded portion. A weighted average is then calculated based on the first probability and the second probability, and the context model is initialized according to the weighted average.

In some examples, the decoder can decode a first weighting factor associated with the first decoded portion and decodes a second weighting factor associated with the second decoded portion. Then, the decoder can calculate the weighted average based on the first weighting factor and the second weighting factor. For example, the decoder can calculate the weighted average by applying the first weighting factor on first probability values indicated by the first context states associated with the first decoded portion, and

applying the second weighting factor on second probability values indicated by the second context states associated with the second decoded portion.

In some examples, in response to the bitstream carrying no weighting factor associated with the first decoded portion, the decoder can determine to use zero weighting factor or a default weighting factor associated with the first decoded portion.

At (S1240), the decoder decodes, the current portion from the bitstream according to the context models that are initialized based on the stored context states associated with the decoded portion.

In some examples, the decoder can decode, from the bitstream, an indicator at a group of pictures (GOP) level. The indicator indicates one or more pictures that are allowed for the context inheriting. The decoder can prioritize a decoding of a first picture in the one or more pictures before a decoding of a second picture not in the one or more pictures. In some examples, the indicator is decoded from a reference picture list structure syntax.

Then, the process proceeds to (S1299) and terminates.

The process (1200) can be suitably adapted. Step(s) in the process (1200) can be modified and/or omitted. Additional step(s) can be added. Any suitable order of implementation can be used.

The techniques described above, can be implemented as computer software using computer-readable instructions and physically stored in one or more computer-readable media. For example, FIG. 13 shows a computer system (1300) suitable for implementing certain embodiments of the disclosed subject matter.

The computer software can be coded using any suitable machine code or computer language, that may be subject to assembly, compilation, linking, or like mechanisms to create code comprising instructions that can be executed directly, or through interpretation, micro-code execution, and the like, by one or more computer central processing units (CPUs), Graphics Processing Units (GPUs), and the like.

The instructions can be executed on various types of computers or components thereof, including, for example, personal computers, tablet computers, servers, smartphones, gaming devices, internet of things devices, and the like.

The components shown in FIG. 13 for computer system (1300) are exemplary in nature and are not intended to suggest any limitation as to the scope of use or functionality of the computer software implementing embodiments of the present disclosure. Neither should the configuration of components be interpreted as having any dependency or requirement relating to any one or combination of components illustrated in the exemplary embodiment of a computer system (1300).

Computer system (1300) may include certain human interface input devices. Such a human interface input device may be responsive to input by one or more human users through, for example, tactile input (such as: keystrokes, swipes, data glove movements), audio input (such as: voice, clapping), visual input (such as: gestures), olfactory input (not depicted). The human interface devices can also be used to capture certain media not necessarily directly related to conscious input by a human, such as audio (such as: speech, music, ambient sound), images (such as: scanned images, photographic images obtain from a still image camera), video (such as two-dimensional video, three-dimensional video including stereoscopic video).

Input human interface devices may include one or more of (only one of each depicted): keyboard (1301), mouse (1302),

trackpad (1303), touch screen (1310), data-glove (not shown), joystick (1305), microphone (1306), scanner (1307), camera (1308).

Computer system (1300) may also include certain human interface output devices. Such human interface output devices may be stimulating the senses of one or more human users through, for example, tactile output, sound, light, and smell/taste. Such human interface output devices may include tactile output devices (for example tactile feedback by the touch-screen (1310), data-glove (not shown), or joystick (1305), but there can also be tactile feedback devices that do not serve as input devices), audio output devices (such as: speakers (1309), headphones (not depicted)), visual output devices (such as screens (1310) to include CRT screens, LCD screens, plasma screens, OLED screens, each with or without touch-screen input capability, each with or without tactile feedback capability—some of which may be capable to output two dimensional visual output or more than three dimensional output through means such as stereographic output; virtual-reality glasses (not depicted), holographic displays and smoke tanks (not depicted)), and printers (not depicted).

Computer system (1300) can also include human accessible storage devices and their associated media such as optical media including CD/DVD ROM/RW (1320) with CD/DVD or the like media (1321), thumb-drive (1322), removable hard drive or solid state drive (1323), legacy magnetic media such as tape and floppy disc (not depicted), specialized ROM/ASIC/PLD based devices such as security dongles (not depicted), and the like.

Those skilled in the art should also understand that term “computer readable media” as used in connection with the presently disclosed subject matter does not encompass transmission media, carrier waves, or other transitory signals.

Computer system (1300) can also include an interface (1354) to one or more communication networks (1355). Networks can for example be wireless, wireline, optical. Networks can further be local, wide-area, metropolitan, vehicular and industrial, real-time, delay-tolerant, and so on. Examples of networks include local area networks such as Ethernet, wireless LANs, cellular networks to include GSM, 3G, 4G, 5G, LTE and the like, TV wireline or wireless wide area digital networks to include cable TV, satellite TV, and terrestrial broadcast TV, vehicular and industrial to include CANBus, and so forth. Certain networks commonly require external network interface adapters that attached to certain general purpose data ports or peripheral buses (1349) (such as, for example USB ports of the computer system (1300)); others are commonly integrated into the core of the computer system (1300) by attachment to a system bus as described below (for example Ethernet interface into a PC computer system or cellular network interface into a smartphone computer system). Using any of these networks, computer system (1300) can communicate with other entities. Such communication can be uni-directional, receive only (for example, broadcast TV), uni-directional send-only (for example CANbus to certain CANbus devices), or bi-directional, for example to other computer systems using local or wide area digital networks. Certain protocols and protocol stacks can be used on each of those networks and network interfaces as described above.

Aforementioned human interface devices, human-accessible storage devices, and network interfaces can be attached to a core (1340) of the computer system (1300).

The core (1340) can include one or more Central Processing Units (CPU) (1341), Graphics Processing Units (GPU) (1342), specialized programmable processing units

in the form of Field Programmable Gate Areas (FPGA) (1343), hardware accelerators for certain tasks (1344), graphics adapters (1350), and so forth. These devices, along with Read-only memory (ROM) (1345), Random-access memory (1346), internal mass storage such as internal non-user accessible hard drives, SSDs, and the like (1347), may be connected through a system bus (1348). In some computer systems, the system bus (1348) can be accessible in the form of one or more physical plugs to enable extensions by additional CPUs, GPU, and the like. The peripheral devices can be attached either directly to the core's system bus (1348), or through a peripheral bus (1349). In an example, the screen (1310) can be connected to the graphics adapter (1350). Architectures for a peripheral bus include PCI, USB, and the like.

CPUs (1341), GPUs (1342), FPGAs (1343), and accelerators (1344) can execute certain instructions that, in combination, can make up the aforementioned computer code. That computer code can be stored in ROM (1345) or RAM (1346). Transitional data can be also be stored in RAM (1346), whereas permanent data can be stored for example, in the internal mass storage (1347). Fast storage and retrieve to any of the memory devices can be enabled through the use of cache memory, that can be closely associated with one or more CPU (1341), GPU (1342), mass storage (1347), ROM (1345), RAM (1346), and the like.

The computer readable media can have computer code thereon for performing various computer-implemented operations. The media and computer code can be those specially designed and constructed for the purposes of the present disclosure, or they can be of the kind well known and available to those having skill in the computer software arts.

As an example and not by way of limitation, the computer system having architecture (1300), and specifically the core (1340) can provide functionality as a result of processor(s) (including CPUs, GPUs, FPGA, accelerators, and the like) executing software embodied in one or more tangible, computer-readable media. Such computer-readable media can be media associated with user-accessible mass storage as introduced above, as well as certain storage of the core (1340) that are of non-transitory nature, such as core-internal mass storage (1347) or ROM (1345). The software implementing various embodiments of the present disclosure can be stored in such devices and executed by core (1340). A computer-readable medium can include one or more memory devices or chips, according to particular needs. The software can cause the core (1340) and specifically the processors therein (including CPU, GPU, FPGA, and the like) to execute particular processes or particular parts of particular processes described herein, including defining data structures stored in RAM (1346) and modifying such data structures according to the processes defined by the software. In addition or as an alternative, the computer system can provide functionality as a result of logic hard-wired or otherwise embodied in a circuit (for example: accelerator (1344)), which can operate in place of or together with software to execute particular processes or particular parts of particular processes described herein. Reference to software can encompass logic, and vice versa, where appropriate. Reference to a computer-readable media can encompass a circuit (such as an integrated circuit (IC)) storing software for execution, a circuit embodying logic for execution, or both, where appropriate. The present disclosure encompasses any suitable combination of hardware and software.

APPENDIX A: ACRONYMS

JEM: joint exploration model
VVC: versatile video coding

BMS: benchmark set
MV: Motion Vector
HEVC: High Efficiency Video Coding
SEI: Supplementary Enhancement Information
VUI: Video Usability Information
GOPs: Groups of Pictures
TUs: Transform Units,
PUs: Prediction Units
CTUs: Coding Tree Units
CTBs: Coding Tree Blocks
PBs: Prediction Blocks
HRD: Hypothetical Reference Decoder
SNR: Signal Noise Ratio
CPUs: Central Processing Units
GPUs: Graphics Processing Units
CRT: Cathode Ray Tube
LCD: Liquid-Crystal Display
OLED: Organic Light-Emitting Diode
CD: Compact Disc
DVD: Digital Video Disc
ROM: Read-Only Memory
RAM: Random Access Memory
ASIC: Application-Specific Integrated Circuit
PLD: Programmable Logic Device
LAN: Local Area Network
GSM: Global System for Mobile communications
LTE: Long-Term Evolution
CANBus: Controller Area Network Bus
USB: Universal Serial Bus
PCI: Peripheral Component Interconnect
FPGA: Field Programmable Gate Areas
SSD: solid-state drive
IC: Integrated Circuit
CU: Coding Unit

While this disclosure has described several exemplary embodiments, there are alterations, permutations, and various substitute equivalents, which fall within the scope of the disclosure. It will thus be appreciated that those skilled in the art will be able to devise numerous systems and methods which, although not explicitly shown or described herein, embody the principles of the disclosure and are thus within the spirit and scope thereof.

What is claimed is:

1. A method of video processing in a decoder, comprising:
 - determining, a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for decoding a current portion of a video sequence carried by a bitstream;
 - decoding, from the bitstream, a flag that indicates whether a CABAC state inheritance is allowed and a syntax value that indicates at least a first decoded portion and a second decoded portion in the video sequence for a context inheriting;
 - when the flag indicates the CABAC state inheritance is allowed and weighting factors of at least the first decoded portion and the second decoded portion are signaled,
 - initializing, context models for the CABAC based on a weighted average of probability values indicated by at least first context states associated with the first decoded portion and second context states associated with the second decoded portion, wherein the weighted average is based on the signaled weighting factors of at least the first decoded portion and the second decoded portion;
 - when the bitstream does not include the weighting factor of the first decoded portion, determining to use a zero

29

weighting factor or a default weighting factor associated with the first decoded portion; and
 decoding, the current portion from the bitstream according to the initialized context models for the CABAC.

2. The method of claim 1, wherein the decoding the syntax value further comprises at least one of:

- decoding the syntax value that indicates a decoded picture for the context inheriting; or
- decoding the syntax value that indicates a decoded slice for the context inheriting.

3. The method of claim 1, wherein the syntax value is decoded from at least one of a picture parameter set (PPS), a picture header, or a slice header.

4. The method of claim 1, wherein one of the first decoded portion and the second decoded portion is a reference that is used for the decoding of the current portion.

5. The method of claim 1, wherein the decoding the syntax value further comprises:

- decoding a first weighting factor associated with the first decoded portion; and
- decoding a second weighting factor associated with the second decoded portion.

6. The method of claim 5, further comprising:

- calculating the weighted average by applying the first weighting factor on first probability values indicated by the first context states associated with the first decoded portion, and applying the second weighting factor on second probability values indicated by the second context states associated with the second decoded portion.

7. The method of claim 1, further comprising:

- decoding, from the bitstream, an indicator at a group of pictures (GOP) level, the indicator indicating one or more pictures that are allowed for the context inheriting; and
- prioritizing a decoding of a first picture in the one or more pictures before a decoding of a second picture not in the one or more pictures.

8. The method of claim 7, wherein the decoding the indicator at the GOP level further comprises:

- decoding the indicator from a reference picture list structure syntax.

9. A method of processing visual media data, the method comprising:

- processing a bitstream that includes the visual media data according to a format rule, wherein
- the bitstream includes a video sequence; and
- the format rule specifies that:
 - a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for decoding a current portion of the video sequence is determined,
 - a flag indicates whether a CABAC state inheritance is allowed,
 - a syntax value indicates at least a first decoded portion and a second decoded portion in the video sequence for a context inheriting,
- when weighting factors of at least the first decoded portion and the second decoded portion are signaled, context models for the CABAC are initialized based on a weighted average of probability values indicated by at least first context states associated with

30

the first decoded portion and second context states associated with the second decoded portion, wherein the weighted average is based on the signaled weighting factors of at least the first decoded portion and the second decoded portion, when the bitstream does not include the weighting factor of the first decoded portion, a zero weighting factor or a default weighting factor associated with the first decoded portion is used, and

the current portion of the video sequence is processed according to the initialized context models for the CABAC.

10. The method of claim 9, wherein the syntax value indicates at least one of:

- a decoded picture for the context inheriting; or
- a decoded slice for the context inheriting.

11. The method of claim 9, wherein the syntax value is included in at least one of a picture parameter set (PPS), a picture header, or a slice header.

12. The method of claim 9, wherein one of the first decoded portion and the second decoded portion is a reference that is used for the decoding of the current portion.

13. A method for video encoding in a video encoder, the method comprising:

- determining, a use of an inherited context initialization in a context-based adaptive binary arithmetic coding (CABAC) for encoding a current portion of a video sequence;
- generating a flag that indicates whether a CABAC state inheritance is allowed and a syntax value that indicates at least a first encoded portion and a second encoded portion in the video sequence for a context inheriting;
- generating weighting factors of at least one of the first encoded portion and the second encoded portion;
- initializing, context models for the CABAC based on a weighted average of probability values indicated by at least first context states associated with the first encoded portion and second context states associated with the second encoded portion, wherein the weighted average is based on the generated weighting factors of at least the first encoded portion and the second encoded portion;
- when the first encoded portion is not associated with one of the at least one generated weighting factors, using a zero weighting factor or a default weighting factor associated with the first encoded portion; and
- encoding, the current portion in a bitstream according to the initialized context models for the CABAC.

14. The method of claim 13, wherein the generating the syntax value further comprises at least one of:

- generating the syntax value that indicates an encoded picture for the context inheriting; or
- generating the syntax value that indicates an encoded slice for the context inheriting.

15. The method of claim 13, wherein the syntax value is encoded in at least one of a picture parameter set (PPS), a picture header, or a slice header.

16. The method of claim 13, wherein one of the first encoded portion and the second encoded portion is a reference that is used for the encoding of the current portion.

* * * * *